

“C++: 31 hours long video of freecodecamp”

***Didn't add basics Like: if,else; Loop,Function, Switch etc.

Features:

Supports data abstraction, means Data abstraction is a fundamental concept in computer science and software engineering that involves hiding the implementation details of data types and exposing only the essential features or behaviour to the outside world. Data abstraction is a powerful concept that facilitates the design, implementation, and maintenance of complex software systems by providing clear interfaces, encapsulating implementation details, promoting modularity, and supporting information hiding.

Statement: A statement is a basic unit of computation in a C++ program. Ends with a semicolon(;).

They are executed from top to bottom when program is run.

stream	Purpose
std::cout	Printing data to the console (terminal)
std::cin	Reading data from the terminal
std::cerr	Printing errors to the console
std::clog	Printing log messages to the console

input For Space separated word



```
int age;
std::string full_name;

std::cout << "Please type in your full name : " << std::endl;
std::getline(std::cin, full_name);
```

Variable: A piece of memory that used to store specific type of data.

Number System representation:

```
int number1 = 15; //Decimal
int number2 = 017; //Octal.
int number3 = 0x0f; //Hexadecimal
int number4 = 0b00001111; //Binary - C++14
```

*** Modifier like *signed*, *unsigned* used integral data type as decimal number/ whole number. Can't use for *float*, *double*(2.0, 2.5).

Float and Double:

Type	Size	Precision	Comment
float	4	7	-
double	8	15	Recommended default
long double	12	> double	

**Precision includes digits before decimal point(.). 12547.325-> 12547 included in the precision.

For Float: 12345.6789-> 89 will be garbage value. Cause precision 7 for Float. May print like: 12345.6725

** float a=123456789-> 89 will be garbage value. May print like: 12345.6745

```
Scientific Notation

double number5 {192400023};
double number6 {1.92400023e8};
double number7 {1.924e8};
double number8 {0.000000000003498};
double number9 {3.498e-11};
```

Floating point number memory representation is not similar to decimal number. It explained in IEEE_754 -> IEEE 754-> Press ctrl & Click

- Remember the suffixes when initializing floating point variables, otherwise the default will be double
- Double works well in many situations, so you will see it used a lot

```
//Declare and initialize the variables
float number1 {1.12345678901234567890f};
double number2 {1.12345678901234567890};
long double number3 {1.12345678901234567890L};
```

```
cout<<setprecision(20);
```

```
int main() {
    double a=12.123456789101111213;
    cout<<setprecision(4)<<fixed<<a<<endl;
```

If we not set precision, the default precision will be 6 digits. (Total digits. Not after decimal point)
 if we use “**fixed**” the with set precision(4), precision will be 4 after decimal point. Above a=12.1234;(ans)
 But not using “fixed” precision will be 4 for total digits. Above a=12.12;(ans)
 used for set precision. Library> #include<iomanip>

suffixes (like f, L): u // unsigned
 ul // unsigned long
 ll // long long

-----xxx-----

****Booleans occupy 1byte/8bits in memory.**

```
bool x=true; // or bool x=1;
bool y=false; // or bool y=0;
cout<<boolalpha; //used for printf true/false instead of 0/1
cout<<x<<" "<<y;
will print true, false.
```

Character:

```
char value = 65 ; // ASCII character code for 'A'
std::cout << "value : " << value << std::endl;
std::cout << "value(int) : " << static_cast<int>(value) << std::endl;
```

Precedence : which operation to do first Associativity : which direction or which order

*** 31/10 means how many times 10 is gonna fit in 31. so ans is 3.*

** Relation Operator: >, <, >=, <=*

** Logical Operator: &&, ||, !*

- ASCII was among the first encodings to represent text in a computer.
- It falls short when it comes to representing languages other than English and a few western languages. Think Arabic, East Asian Languages like Japanese, Chinese ,...
- There are better ways to represent text that is meant to be seen in different languages, one of the most common being Unicode
- The details of Unicode are out of scope for this course, just know that it's a robust way to represent text in different languages for a computer

Manipulator: <https://en.cppreference.com/w/cpp/io/manip> //for more

** **std::flush:** when we print something, it does not go directly to the terminal. It store somewhere called "buffer". When buffer is full/ complete it goes to terminal. If use std::flush data directly goes to console/terminal instead of goes to buffer.*

std::setw()

```
std::cout << "Unformatted table : " << std::endl;
std::cout << "Daniel" << " " << "Gray" << " 25" << std::endl;
std::cout << "Stanley" << " " << "Woods" << " 33" << std::endl;
std::cout << "Jordan" << " " << "Parker" << " 45" << std::endl;
std::cout << "Joe" << " " << "Ball" << " 21" << std::endl;
std::cout << "Josh" << " " << "Carr" << " 27" << std::endl;
std::cout << "Izaiah" << " " << "Robinson" << " 29" << std::endl;

std::cout << std::endl;
std::cout << "Formatted table : " << std::endl;

std::cout << std::setw(10) << "Lastname" << std::setw(10) << "Firstname" << std::setw(5) << "Age" << std::endl;
std::cout << std::setw(10) << "Daniel" << std::setw(10) << "Gray" << std::setw(5) << "25" << std::endl;
std::cout << std::setw(10) << "Stanley" << std::setw(10) << "Woods" << std::setw(5) << "33" << std::endl;
std::cout << std::setw(10) << "Jordan" << std::setw(10) << "Parker" << std::setw(5) << "45" << std::endl;
std::cout << std::setw(10) << "Joe" << std::setw(10) << "Ball" << std::setw(5) << "21" << std::endl;
std::cout << std::setw(10) << "Josh" << std::setw(10) << "Carr" << std::setw(5) << "27" << std::endl;
std::cout << std::setw(10) << "Izaiah" << std::setw(10) << "Robinson" << std::setw(5) << "29" << std::endl;
```

Unformatted table :
Daniel Gray 25
Stanley Woods 33
Jordan Parker 45
Joe Ball 21
Josh Carr 27
Izaiah Robinson 29

Formatted table :
Lastname Firstname Age
Daniel Gray 25
Stanley Woods 33
Jordan Parker 45
Joe Ball 21
Josh Carr 27
Izaiah Robinson 29

**setw() : set width*

cout<<right; // printf from right.

Left for left alignment

Head	Head
1	2

cout<<setfill('-') // blank space fill with '-'

```
Lastname-----Firstname-----Age-----
Daniel-----Gray-----25-----
Stanley-----Woods-----33-----
Jordan-----Parker-----45-----
Joe-----Ball-----21-----
Josh-----Carr-----27-----
Izaiah-----Robinson-----29-----
```

**for finding max min numeric number limits data type can hold.*

Need this> #include<limits>

https://en.cppreference.com/w/cpp/types/numeric_limits

```
main.cpp > main()
1  #include <iostream>
2  #include<iomanip>
3  #include<limits>
4  using namespace std;
5  #define NL printf("\n")
6  #define nl endl
7
8  int main(){
9      int a=300, b=100;
10     cout<<numeric_limits<int>::min()<<nl;
11     cout<<numeric_limits<int>::max()<<nl;
12     NL;
13     cout<<numeric_limits<unsigned int>::min()<<nl;
14     cout<<numeric_limits<unsigned int>::max()<<nl;
15 }
```

#include<cmath> : abs(), pow(), ceil(), log(), sqrt(), sin(), tan() etc

<https://en.cppreference.com/w/cpp/header/cmath>

for `log()`: `log(10)` means $\log_e(10)$. So have to fix the base as $\log_{10}(10)$. $E=2.71..$

* `round()`: 3.5 will make 4, and 3.49 will make 3.

Integral types less than 4 bytes in size don't support arithmetic operations

if we take data type less than 4 byte and perform arithmetic operation compiler automatically convert it to 4 byte. This behavior also present on other operator like bitwise operator (`>>`, `<<`)

flow control: if else, switch, ternary operator.

* **Switch**: if we not use "Break", the case which match, after that all case will execute and print every case value.

* we can use `int`, `char`, `double`, **enum** etc but not string as case.

* If we use "const" before array. We cant modify array elements.

* `a[] = {2,3,7,5,2,3}`; `size(a)` return the size of array. /Or `sizeof(a)/sizeof(a[0])`;

Static Variable: A static variable initialize only once and retains its value between function calls / across different scopes, instead of being destroyed and recreated each time.

```
#include <iostream>
using namespace std;

void counter() {
    static int count = 0; // initialized only once
    count++;
    cout << "Count: " << count << endl;
}

int main() {
    counter(); // Count: 1
    counter(); // Count: 2
    counter(); // Count: 3
    return 0;
}
```

- is created only once
- exists for the entire lifetime of the program
- is not destroyed when it goes out of scope

Static Global Variable: As we know global variables are accessible from other .cpp file. In case of static global declaration, the variable is only visible within that translation unit (source file), not to other .cpp files.

```
static int globalCount = 0; // internal linkage, only visible in this file

void increment() {
    globalCount++;
}
```

Enum: An **enum** is a special type that represents a group of constants (unchangeable values).

To create an enum, use the **enum** keyword, followed by the name of the enum, and separate the enum items with a comma.

```
enum Level {
    LOW,
    MEDIUM,
    HIGH
};
```

Enum is short for "enumerations", which means "specifically listed".

To access the enum, you must create a variable of it.

Inside the **main()** method, specify the **enum** keyword, followed by the name of the enum (**Level**) and then the name of the enum variable (**myVar** in this example):

```
enum Level myVar;
```

By default, the first item (**LOW**) has the value **0**, the second (**MEDIUM**) has the value **1**, etc.

If you now try to print myVar, it will output **1**, which represents **MEDIUM**:

```
int main() {
    // Create an enum variable and assign a value to it
    enum Level myVar = MEDIUM;

    // Print the enum variable
    cout << myVar;

    return 0;
}
```

As you know, the first item of an enum has the value 0. The second has the value 1, and so on. To make more sense of the values, you can easily change them:

```
enum Level {
    LOW = 25,
    MEDIUM = 50,
    HIGH = 75
};

int main() {
    enum Level myVar = MEDIUM;
    cout << myVar; // Now outputs 50
    return 0;
}
```

Note that if you assign a value to one specific item, the next items will update their numbers accordingly:

```
enum Level {
    LOW = 5,
    MEDIUM, // Now 6
    HIGH // Now 7
};
```

Example:

```
enum Level {
    LOW = 1,
    MEDIUM,
    HIGH
};

int main() {
    enum Level myVar = MEDIUM;

    switch (myVar) {
        case 1:
            cout << "Low Level";
            break;
        case 2:
            cout << "Medium level";
            break;
        case 3:
            cout << "High level";
            break;
    }
    return 0;
}
```

Why And When To Use Enums?

Enums are used to give names to constants, which makes the code easier to read and maintain. Use enums when you have values that you know aren't going to change, like month days, days, colours, deck of cards, etc.

Pointer

Pointer is special kind of variable.

```
int* int_num{}; or int* int_num; // will point to a integer type variable //initialise with null
// pointer(nulptr)

double* frac_num{}; // will point to a double type variable

int * int_num{}; // this initialisation with {} is going to initialise with special address means it is not
//pointing any variable. Means initialize with nullptr

int * int_num{nullptr}; // this pointer not pointer anywhere

** pointer of int, double, char etc all are same size. Cause they only store address.
```

Char *ptr{"Hello world"}; // Pointer will point to 1st character. Some compiler will not compile(MSVC). GCC will give warning and compile. A whole string is assigning to char type pointer. **See:10:17:00**

```
Cout<<ptr; // will print whole string.
Cout<<*ptr; // print 1st character 'H';
*ptr='B'; // this may give error. Cause compiler will think it as const char array.
```

If want to modify. Don't use character pointer, use array like: **char msg[10]="Hello world";**
or use **const char *ptr{"Hello world"};** // no warning. Can't modify also.

Dereferencing pointer: reading something(value) on the address of the pointer. `Cout<<*ptr;`

string with pointer: `char* p_msg= "Hello World!";` // the pointer will point to the 1st character of string

*this will compile with warning. Use **const char* p_msg= "Hello World!";**

`printf p_msg` will print whole string. But using dereferencing will print 1st character(*p_msg).

*** without **const** it gives warning/refuse to compile cause compiler is going to convert string into char array of constant char. What we are using is points to that is not a const char pointer. So pointer here might be used to try or modify data. That's why it refuse unless using **const**.

Check at 10:17min

```
int *ptr; // contain junk address
int a=12;
*ptr=&a;
```

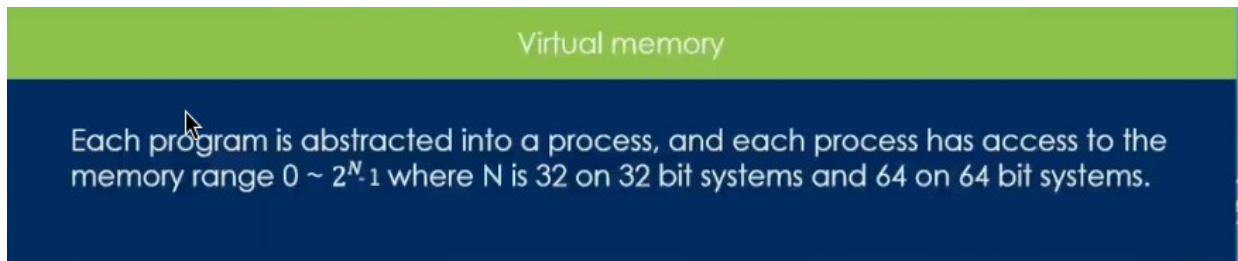
uninitialized pointer contain junk address. Assigning a value to it(*ptr=12) May cause error. Could be point to a memory which is used by OS. May cause disaster.

Use this: `int a; int *ptr=&a;`

```
//Initializing pointer to null
//int *p_number3{nullptr}; // Also works
int * p_number3 {}; // Initialized with pointer equivalent of zero : nullptr
// A pointer pointing nowhere
// *p_number3 = 33; // Writting into a pointer pointing nowhere : BAD, CRASH
```

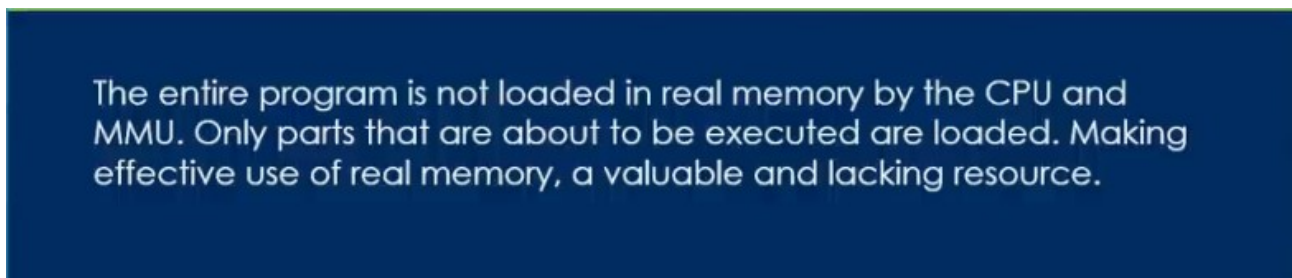
Memory Map

When we run a program it runs on RAM. Various program of OS or other is running on memory.



This process thinks it own $0 \sim 2^N$ (2 pow N) amount of memory which is virtual memory.

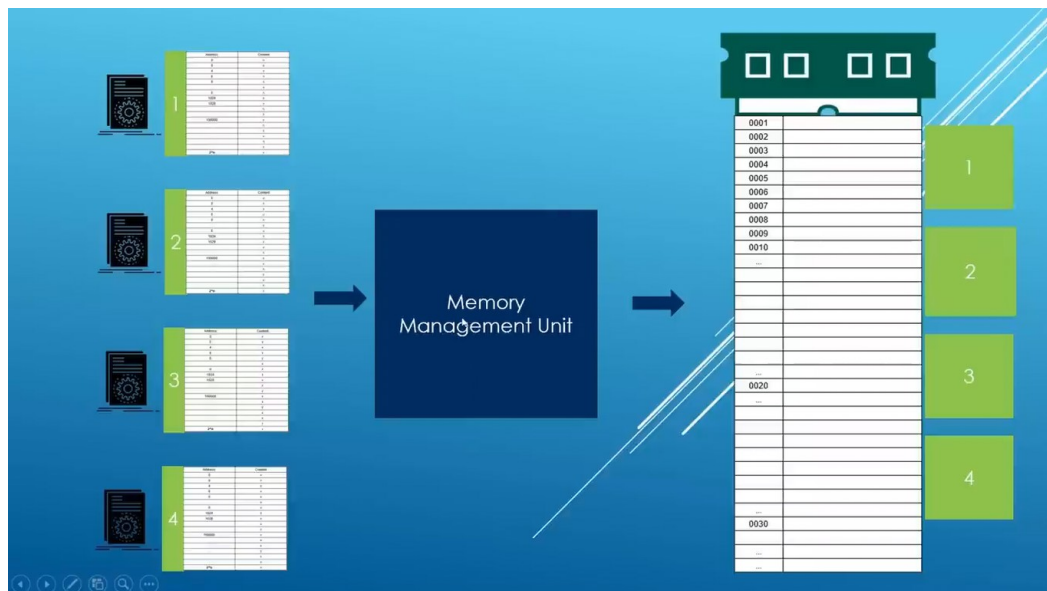
When we run a program it is going to go through a CPU section called memory management unit(MMU).



Part that are likely not to be used are discarded from the RAM.

MMU really does is, helps us mapping between the memory map in ur program and the real thing we have in RAM.

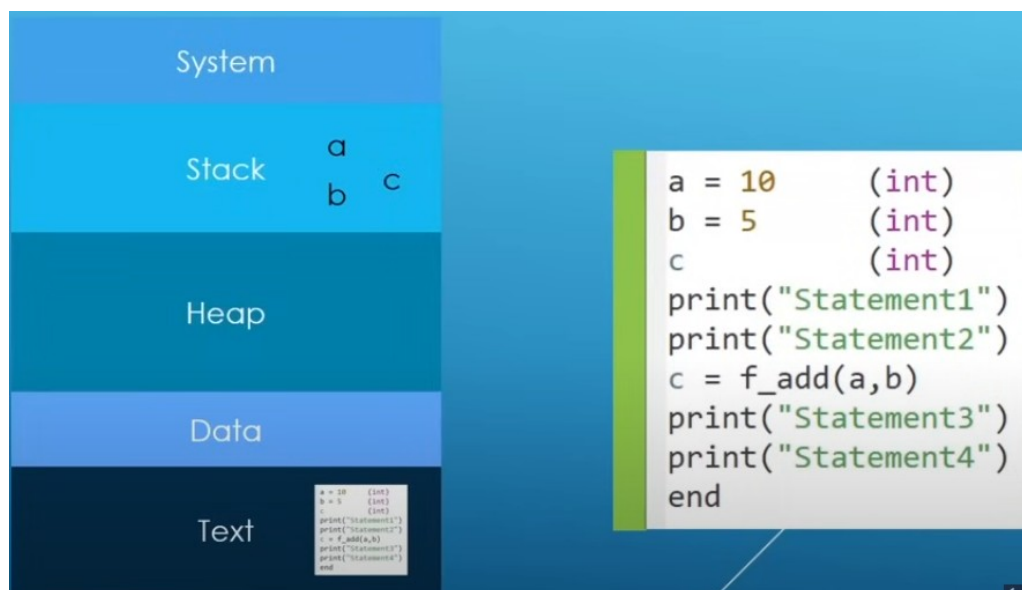
If we run few program, they are going to go through MMU and MMU is going assign them real section on RAM



* Since program thinks it $2^n - 1$ memory, The MMU is going to transform between the idea the program has and the RAM we have (assigned memory by MMU).

* The memory map/Structure of program is standard format defined by OS. That's why we can't run directly window program on Linux.

* Memory map is divided into a lot of parts



* **STACK:** Local variable stores on stack section.

* print, statement, function calls others store on stack section.

* **TEXT:** Actual binary load on Text so that CPU can execute it.

* **HEAP:** Additional memory when we run out of stack memory also to make things better for program, Used for run time.

Dynamic Memory Allocation

Stack	Heap
• Memory is finite • The developer isn't in full control of the memory lifetime • Lifetime is controlled by the scope mechanism	• Memory is finite • The developer is in full control of when memory is allocated and when it's released • Lifetime is controlled explicitly through new and delete operators

2nd point- full Control: when declare a variable `a=23`; in stack it remove when scope is over. Developer doesn't have full control. But in heap developer have full control when a variable comes to work and dies.

>> 10:41 min

Allocate dynamic memory through 'new'

```
//Initialize the pointer with dynamic memory. Dynamically allocate
//memory at run time and make a pointer point to it

int *p_number4{nullptr};
p_number4 = new int;    // Dynamically allocate space for a single int on the heap
                        // This memory belongs to our program from now on. The system
                        // can't use it for anything else, untill we return it.
                        //After this line executes, we will have a valid memory location
                        // allocated. The size of the allocated memory will be such that it
                        // can store the type pointed to by the pointer

*p_number4 = 77; // Writting into dynamically allocated memory
std::cout << std::endl;
std::cout << "Dynamically allocating memory : " << std::endl;
std::cout << "*p_number4 : " << *p_number4 << std::endl;
```

**set up a pointer which point to heap memory.*

After initializing a pointer with `nullptr`. When `p_number4=new int` execute the OS is allocate a memory on heap. Variables are usually stores on stack section. It removes when variable containing scope will over. But if we allocate a memory on heap by "`p_number4=new int`";. This memory will live though its scope is over. It will stay until return it to operating system. To return:

Releasing and Resetting

```
int *p_number4{nullptr};  
p_number4 = new int;  
  
/*  */  
  
delete p_number4;  
p_number4 = nullptr;
```

Use 'delete' to return memory to the operating system. After return reset pointer to 'nullptr' is good to make it clear that no valid data pointer is pointing.

* Using 'delete' remove the allocated heap memory which is pointed. Not the pointer. If pointing to 'nullptr', delete will do nothing.

*** Always remember to release memory.

***Dangling Pointer:** A pointer that doesn't point to a valid memory address. Trying to dereferencing and using a dangling pointer will results in undefined behaviour.

How dangling pointer create:

1. Uninitialized pointer.
2. Delete pointer
3. Multiple pointer points to same memory.

Solution:

1. Initialize pointer. Either with valid address or nullptr.
2. Reset pointer after delete. (Either with valid address or nullptr.)
3. For multiple pointer point to same address, make sure master pointer is clear/ reset.

*** Always check if pointer is nullptr or not by if-else.

'New' fails:

When allocating an array with pointer with huge size(10000000000000000000). It may fails and program can crash. We can use **exception mechanism or 'nothrow'** to prevent crashing.

```
int *ptr=new int[1000000000000];  
for(size_t i=0; i<10000000; i++){  
    int *ptr= new int[10000];  
}
```

Both allocation (with for loop or without) fail.

*** Solve with 'exception mechanism':**

```
for(size_t i=0; i<100; i++){
    try {
        int *ptr= new int[10000000000000];
    } catch (exception& ex) {
        cout<<"Problem: " <<ex.what();NL;
    }
}
```

With 'exception' we can catch the problem. What is called 'handle' in the problem. Suppose we are going to set up color and color fails. UI(interface) may show black and white. But program will keep running "what()" function will show the error.

***with 'nothrow':** If "new" fails, we are going to get "nullptr" stored.

```
for(size_t i=0; i<100; i++){
    int *ptr= new(nothrow) int[10000000000000];
    if(ptr!=nullptr) cout<<"Memory allocated";
    else cout<<"Memory not allocated";
    NL;
}
```

For clear understanding see

<https://www.youtube.com/watch?v=uoCuMTzD9AE&list=PLgH5QX0i9K3q0ZKeXtF--CZOPdH1sSbYL&index=90> // anisul islam lecture 92. Exception handling

NULL pointer safety:

```
int *p_number{}; // Initialized to nullptr
```

check if null or not.

```
if(!(p_number==nullptr)){
    std::cout << "p_number points to a VALID address : " << p_number << std::endl;
}else{
    std::cout << "p_number points to an INVALID address." << std::endl;
}
```

Or,

```
if(p_number){
    std::cout << "p_number points to a VALID address : " << p_number << std::endl;
}else{
    std::cout << "p_number points to an INVALID address." << std::endl;
}
```


Pointer address implicitly converted into boolean. (null==0).

* After use delete set pointer to nullptr for safety.

Memory Leaks:

Reassignment of stack address to active dynamic address pointer

```
int *p_number {new int{67}}; // Points to some address, let's call that address1
//Should delete and reset here

int number{55}; // lives at address2

p_number = &number; // Now p_number points to address2 , but address1 is still in use by
                    // our program. But our program has lost access to that memory location.
                    //Memory has been leaked.
```

```
//Double allocation
int *p_number1 {new int{55}};

//Use the pointer

//Should delete and reset here.

p_number1 = new int{44}; // memory with int{55} leaked.
```

while 2nd allocation, pointer changes its pointing location to 2nd memory. Don't have access of 1st memory(55). Should delete 1st then allocate 2nd.

Pointer in local scope

```
#include <iostream>

int main(int argc, char **argv)
{
    ...

    {
        int *p_number2 {new int{57}};
    }
    //Memory with int{57} leaked.

    return 0;
}
```

After local scope is over, pointer is gonna die, but allocated memory will remain and loose access.
These memory leaks may causes program crash.

Dynamic array: Array stores on the heap.

Array dynamic allocation

```
size_t size{10};

//Different ways you can declare an array
//dynamically and how they are initialized

double *p_salaries { new double[size]}; // salaries array will
//contain garbage values
int *p_students { new(std::nothrow) int[size]{} }; // All values initialized to 0
double *p_scores { new(std::nothrow) double[size]{1,2,3,4,5}}; // Allocating memory space
// for an array of size double
//vars. First 5 will be initialized
//with 1,2,3,4,5, and the
//rest will be 0's.
```

```
//nullptr check and use the allocated array
if(p_scores){
    //Print out elements. Can use regular array access notation, or pointer arithmetic
    for( size_t i{}; i < size ; ++i){
        std::cout << "value : " << p_scores[i] << " : " << *(p_scores + i) << std::endl;
    }
}
```

Release Memory:

```
delete[] p_scores;
p_scores = nullptr;

delete[] p_students;
p_students = nullptr;

delete[] p_salaries;
p_salaries = nullptr;
```

Dynamic arrays are created at run time not compile time.

Pointers and arrays are different

```
//Pointers initialized with dynamic arrays are different from arrays :  
//std::size doesn't work on them, and they don't support range based for loops  
  
double *temperatures = new double[size] {10.0,20.0,30.0,40.0,50.0,60.0,70.0,80.0,90.0,100.0};  
  
//std::cout << "std::size(temperatures) : " << std::size(temperatures) << std::endl; //Error  
  
//Error : temperatures doesn't have array properties that are needed for  
// the range based for loop to work.  
for (double temp : temperatures){  
    std::cout << "temperature : " << temp << std::endl;  
}  
  
//We say that the dynamically allocated array has decayed into a pointer
```

Explanation:

You're creating a **pointer**, not a real array type. It points to a memory block of 5 integers on the heap — but the compiler **doesn't know the size** of that memory.

❌ No Range-Based Loops with `new[]`

cpp

Copy Edit

```
int* arr = new int[5];  
for (int x : arr) // ❌ Error: pointer has no size info!
```

You must use:

cpp

Copy Edit

```
for (int i = 0; i < 5; ++i)  
    cout << arr[i];
```

Reference: A reference is an alias (another name) for an existing variable. It does not create a new variable or occupy memory, it just gives another way to access the same memory. Doesn't hold addresses as pointer.

```
int s=10;  
int &ref=s;  
printf("%d %p %p %d\n", s, &s, &ref, ref);  
ref=100;  
printf("%d %d\n", s, ref);  
s=1000;  
printf("%d %d\n", s, ref);
```

```
int x = 10;  
int& ref = x;
```

What happens in memory?

Suppose `x` is stored like this:

Address	Value
1000	10

After creating the reference:

```
<> C++  
int& ref = x;
```

The memory is still:

Address	Value
1000	10

There is **no extra memory** allocated for `ref`.

Instead, the compiler simply treats `ref` as **another name for the object at address 1000**.

So these are exactly equivalent:

```
<> C++  
x = 20;
```

and

```
<> C++  
ref = 20;
```

Both modify the same integer at address `1000`.

If we change reference value or variable value, both contribute same changes.

*** "&ref" and "&s" both has same memory address.*

References	Pointers
• Don't use dereferencing for reading and writing • Can't be changed to reference something else • Must be initialized at declaration	• Must go through dereference operator to read/write through pointed to value • Can be changed to point somewhere else • Can be declared un-initialized (will contain garbage addresses)

```
//Declare pointer and reference
double double_value {12.34};

double& ref_double_value {double_value}; // Reference to double_value
double* p_double_value {&double_value}; //Pointer to double_value

//Reading
std::cout << "double_value : " << double_value << std::endl;
std::cout << "ref_double_value : " << ref_double_value << std::endl;
std::cout << "p_double_value : " << p_double_value << std::endl;
std::cout << "*p_double_value : " << *p_double_value << std::endl;
```

Reassign a Pointer to others variable, but reference can't.

```
int s=10;
int &ref=s;
int ss=12;
ref=ss;
printf("%d %d\n", s, ref);
```

Here 'S and ref' value will changes to 12.

Use case: **if want to modify original variable inside a function. This save memories.*

```
void square(int &x) {
    x = x * x;
}

int main() {
    int a = 5;
    square(a); // modifies 'a'
    cout << a; // Output: 25
}
```

* Return reference by function to a local or global variable.

```
vector<int> v = {1, 2, 3};

for (int &x : v) {
    x += 10; // modifies original vector
}
```

- Without `&`, you'd just be modifying a copy.

References are somewhat like const pointers

```
//References behave like constant pointers, but they have
//a much friendlier syntax as they don't require dereferencing
//to read and write through referenced data.
```

```
double *const const_p_double_value {&double_value};

const_p_double_value = &other_double_value; // Error
```

* Using "const" before reference makes the variable unchangeable. This constant only applies to reference. Not variable. Can't changes variable value with "const reference". But variable itself can changes its value.

```
int s=10;
const int &ref=s;
int ss=12;
// ref=ss; // show error, cause "const"
s=12;
printf("%d %d\n", s, ref);
```

Value will changes to 12.

*** size(), sizeof() function return the size. But for character array it includes 'null'. But for string size() don't count null.

*** strlen() is used for character array. Not for string.

```
char *a="asdf"; // this is string literal. Not modifiable.
```


`Char a[]="asdf"; // this is character array. Modifiable. Use stack memory.`

Using "const" is safe in string literal.

***String literal is actually a character array. When we assign it to "const char *", it automatically converts into a pointer to 1st element. String has not fixed amount of memory, cause it internally implemented as a class and it stores its data as **const char pointer**.

`Const char *msg="Hello I am here.";`

Declaration	Modifiable?	Memory Type	Notes
<code>char* s = "text"</code>	✗ No	Read-only segment	Undefined behavior to change
<code>const char* s = "text"</code>	✗ No	Read-only segment	✓ Safe and correct
<code>char s[] = "text"</code>	✓ Yes	Stack	Literal copied into array
<code>std::string s = "text"</code>	✓ Yes	Heap	Best practice in C++

Character array lives on read only memory. Can't modify.

swap 2 number.

1. `a=a+b; b=a-b; a=a-b;`

2. `a=a^b; b=a^b; a=a^b;`

Returning from function as Reference: Usually function returns values(int, char etc). But sometime compilers are smart enough that that return reference instead of values. Avoid copies. See below

In modern compilers, return by value is commonly optimized out by the compiler when possible and the function is modified behind your back to return by reference, avoiding unnecessary copies!

Example:

```
string add_str(string s1,string s2){
    string res=s1+s2;
    cout<<res<<" "<&res;NL;
    return res;
}

int main(){
    string s1="Hello ", s2="World!";
    string res=add_str(s1,s2);
    cout<<res<<" "<&res;NL;
```

Output:

```
› g++ proj.cpp && ./a.out
Hello World! 0x7ffe27076d50
Hello World! 0x7ffe27076d50
```

Here both addresses are same. Its returns the reference. Using reference mechanism.

Function Overloading

Function Overloading: Means we can declare multiple function with same name in the same scope, but with different parameter lists. Like parameter type(int, double, float).

Below All allowed.

```
int max(int a, int b){
    int sum=a+b;
    return sum;
}
double max(double a, int b){
    double sum=a+b;
    return sum;
}
```

```
int max(int a, int b){
    int sum=a+b;
    return sum;
}
int max(double a, int b){
    double sum=a+b;
    return sum;
}
```

```

int max(int a, int b, int c){
    int sum=a+b+c;
    return sum;
}

int max(int aa, int bb){
    double sum=aa+bb;
    return sum;
}

```

Not allowed.(below)

```

int max(int a, int b){
    int sum=a+b;
    return sum;
}

int max(int a, int b){
    double sum=a+b;
    return sum;
}

```

```

int max(int a, int b){
    int sum=a+b;
    return sum;
}

int max(int aa, int bb){
    double sum=aa+bb;
    return sum;
}

```

```

int max(int a, int b){
    std::cout << "int overload called" << std::endl;
    return (a>b)? a : b;
}

double max(double a, double b){
    std::cout << "double overload called" << std::endl;
    return (a>b)? a : b;
}

```

When **'int'** type variable is passed as argument "int overload will called". Same as double and others.

lambda Function:

A mechanism to set up anonymous functions (without names). Once we have them set up, we can either give them names and call them , or we can even get them to do things directly.

Lambda function signature

```

+Lambda function signature :
                                [capture list] (parameters) ->return type{
                                    // Function body
                                }

                                [](){
std::cout << "Hello World!" << std::endl;
};

```

* Return type is not important. If keep blank, compiler is gonna deduce its type by itself.

* Use ';' after function body. Because lambda function is a statement.

```

int main(){
    auto fun=[](int a, int b)->int{
        printf("jhbqvfn");
        return a+b;
    };
    printf("%d\n", fun(2,1));
    //Or call without name below
    [](){
        printf("sdfg");
    }();
}

```

```

[](double a, double b){
    std::cout << "a + b : " << (a + b) << std::endl;
}(12.1,5.7);

```

```

int main(){
    auto fun=[](int a, int b)->int{
        printf("jhbgvf\n");
        return a+b;
    }(2,1);
    printf("%d\n", fun);
}

```

Here if lambda function return something, it going to assign to variable 'fun'.

Capture lists on lambda function: Capture list is a part of lambda function inside the square brackets which tells the lambda function, which variable from the surrounding scope it can use and how (like using a copy of a variable or references).

* When a lambda function capture values, it made a copy of that variable on the memory. So if that variable is changed later it will not effect on that lambda function. Lambda function retain the old value unless we use variable as reference. See below...

```

int a=12;
auto fun=[a]() {
    printf("Inner value: %d\n", a);
};
for(int i=0; i<5; i++){
    printf("Outer value: %d\n", a);
    fun();
    a++;
}

```

```
› g++ proj.cpp && ./a.out
Outer value: 12
Inner value: 12
Outer value: 13
Inner value: 12
Outer value: 14
Inner value: 12
```

See when we use variable as reference:

```
int a=12;
auto fun=[&a]() {
    printf("Inner value: %d\n", a);
};
for(int i=0; i<3; i++){
    printf("Outer value: %d\n", a);
    fun();
    a++;
}
```

```
› g++ proj.cpp && ./a.out
Outer value: 12
Inner value: 12
Outer value: 13
Inner value: 13
Outer value: 14
Inner value: 14
```

If we print the addresses, we can clearly see that both (variable *a*, and lambda function variable *a*) variable have different addresses.

*** "[=]" using this as capture list it will grab all variable from the surrounding scope. (To capture value)

*** "[#]" using this as capture list it will grab all variable from the surrounding scope. (To capture as references)

*** using reference, all have same addresses.

Function Template:

Function Template by value: Function Template is a mechanism in c++ to set up a blueprint for functions, But compiler going generate the actual code when it sees the function called. Means to avoid code repetition.

```
int max(int a, int b){
    return (a>b)? a : b;
}

double max(double a, double b){
    return (a>b)? a : b;
}
```

Here there are multiple function overload. They are doing the work. To minimise this type of multiple overload of same work. We can use Template.

```
→ template <typename T> T maximum(T a,T b);

int main(int argc, char **argv)
{
    int a{10};
    int b{23};

    double c{34.7};
    double d{23.4};

    std::string e{"hello"};
    std::string f{"world"};

    std::cout << "max(int) : " << maximum(a,b) << std::endl; // int version created
    std::cout << "max(double) : " << maximum(c,d) << std::endl; // double version created
    std::cout << "max(string) : " << maximum(e,f) << std::endl; // string version created

    /* ... */

    return 0;
}

template <typename T> T maximum(T a,T b){
    return (a > b) ? a : b ; // a and b must support the > operator. Otherwise, hard ERROR
}
```

“T” is a place holder for the data type function is going to receive as argument. But all argument data type **have to be same (there is a room for not same data type)**. When compiler see, template function called. It going to look at the data type of arguments which are passing to function. Then compiler generate same function with that data type only when the function is called.

** can also pass string by argument.

Function templates are not c++ code. It just a function blueprint.

- Function templates are just blueprints. They're not real C++ code consumed by the compiler. The compiler generates real C++ code by looking at the arguments you call your function template with
- The real C++ function generated by the compiler is called a template instance
- A template instance will be reused when a similar function call (argument types) is issued. No duplicates are generated by the compiler

If data types are not same passing as argument, we can explicitly set the type with <double>. This basically tells the compiler to generate double/int etc. template instance function for this calling. And implicitly convert other type to determined type. In below example "a" variable int type and will convert to double.

*** We can see this internal conversion of function template on cppinsights.io.*

```
template<typename T> T maxii(T a, T b){
    return a+b;
}

int main(){
    int a=12;
    int b=34;
    double c=12.22;
    auto re=maxii<double>(a,c);
    cout<<re;
```

This will give no error. Cause we explicitly tell to generate a double type template instance function. Otherwise, different type will not accept. Will throw an error.

If we use sizeof() to see size of the "re" variable. We can see is it double, int etc.

Template type parameter by references: Recall references procedure, template procedure. All same concept.

```
template<typename T> T maxii(const T& a, const T& b){
    return a>b?a:b;
}

int main(){
    int a=12;
    int b=34;
    int re=maxii(a,b);
    cout<<re<<endl;
}
```

```
template <typename T> const T& maximum(const T& a, const T& b); // Declaration
template <typename T> T maximum(T a, T b); // Declaration
```

function overloaded. Cause calling

maximum(a,b); // this is used for both template with value and with reference. That's why get confused.

Template Specialization: This is specially for “const char pointer” like:

const char* x=“asdf”;

for const char pointer regular method will not work. There is a special template mechanism for it.

```
//Template specialization
template <>
const char* maximum<const char*> (const char* a , const char* b);
```

This does not compare like others in template. See 18:50 Hr

Instead it use c++ template library function “strcmp” to compare. See on [www.cppreference.com](https://en.cppreference.com/w/c/string/byte/strcmp) about strcmp.

In order to use template specialization we have to declare primary template like below...

Specializing maximum for const char*

```
template <>
const char* maximum<const char*> (const char* a , const char* b){
    //std::strcmp doc : https://en.cppreference.com/w/c/string/byte/strcmp
    return (std::strcmp(a,b) > 0) ? a : b ;
}
```

Return value

Negative value if lhs appears before rhs in lexicographical order.

Zero if lhs and rhs compare equal.

Positive value if lhs appears after rhs in lexicographical order.

In order to use template specialization we have to declare primary template like below...

```
template<typename T> T maxii(const T a, const T b){  
    return a>b?a:b;  
}
```

Whole code:

```
template<typename T> T maxii(const T a, const T b){  
    return a>b?a:b;  
}  
template<  
const char* maxii<const char*>(const char* a, const char* b){  
    return (strcmp(a,b)>0)?a:b;  
}  
int main(){  
    int a=12;  
    int b=34;  
    const char* x="dasdd";  
    const char* y="ebaaa";  
    const char* re=maxii(x,y);  
    cout<<re<<endl;  
}
```

C++ 20:

Concept: Concept is a mechanism to set up constrain or restriction on template parameter of our function template. For example we can set that function to be called only integer and we call it something which isn't an integer, it will give a compiler error.

```
template<typename T>
requires integral<T>
T add(const T a, const T b){
    return a+b;
}

int main(){
    int a=98, b=100;
    auto rs=add(a,b);
    cout<<rs;
}
```

If we set double instead of int, this will compiler error. Cause we set “integral” type to accept as argument in function template.

These concepts are introduced in c++20. So use “-std=c++20” during compiling.

```
~/code via C v15.2.1-gcc
➤ g++ -std=c++20 proj.cpp && ./a.out
```

Concepts are introduced in c++20. Before that we can use this(below) inside function template. And set a custom message if condition does not meet.

```
➔ static_assert(std::is_integral<T>::value , "Must pass in an integral argument");
```

Type Traits: Intro on C++11. A type trait is a small template “tool” that tells you something about a type, or transforms a type, at compile time. On above we are checking “T” is an integral or not by “is_integral<>”. Boolean value. Also Use this as requires. See below

Syntax1 : using type traits

```
template <typename T>
requires std::is_integral_v<T> // Using a type trait
T add (T a, T b){
    return a + b;
}
```

We can use type traits on **requires**.

```

template<typename T>
requires is_floating_point_v<T>
T add(T a, T b){
    return a+b;
}

int main(){
    double a=12.5;
    double b=12;
    auto rs=add(a,b);
    cout<<rs;
}

```

Some Type Traits:

<code>is_void</code> (C++11)	checks if a type is <code>void</code> (class template)
<code>is_null_pointer</code> (C++11)(DR*)	checks if a type is <code>std::nullptr_t</code> (class template)
<code>is_integral</code> (C++11)	checks if a type is an integral type (class template)
<code>is_floating_point</code> (C++11)	checks if a type is a floating-point type (class template)
<code>is_array</code> (C++11)	checks if a type is an array type (class template)
<code>is_enum</code> (C++11)	checks if a type is an enumeration type (class template)
<code>is_union</code> (C++11)	checks if a type is a union type (class template)
<code>is_class</code> (C++11)	checks if a type is a non-union class type (class template)
<code>is_function</code> (C++11)	checks if a type is a function type (class template)
<code>is_pointer</code> (C++11)	checks if a type is a pointer type (class template)

Some ways to declare concepts:

Syntax1

```

template <typename T>
requires std::integral<T>
T add (T a, T b){
    return a + b;
}

```

Syntax4

```

→ template <typename T>
    T add (T a, T b) requires std::integral<T>{
        return a + b;
    }

```

Syntax 3 is only allowed for, when we use “auto”.(Below)

```
Syntax3

auto add (std::integral auto a, std::integral auto b){
    return a + b;
}
```

```
Syntax2

template <std::integral T>
+ T add (T a, T b){
    return a + b;
}
```

Example for Type Trait:

A type trait is a template utility in the C++ standard library (in <type_traits>) that allows you to query information about a type or transform a type at compile time.

Think of type traits as little “questions” or “tools” about types:

- 1. Is this type an integer?*
- 2. Is this type const-qualified?*
- 3. What happens if I remove a pointer from this type?*
- 4. Are these two types the same?*

```
template <typename T>
void print(const T& value) {
    if constexpr (std::is_integral<T>::value) {
        cout << value << " is an integral type\n";
    } else {
        cout << value << " is NOT an integral type\n";
    }
}

int main() {
    print(42);    // integral
    print(3.14);  // not integral
}
```

Here the compiler removes the unused branch at compile time → no runtime cost.

Here without “constexpr” if is checked at run time. So both branch(if, else) must compile because compiler doesn’t know which condition will be true.

But with “constexpr”

** if constexpr means the compiler chooses the branch at compile time.*

* The unused branch is discarded before code generation — it's as if it never existed. Means for ("print(42)") compiler will keep only if part. And for ("print(3.14)") compiler will keep else part.

```
f(42); // T = int → std::is_integral<int>::value == true
// Compiler keeps only the "integral" branch

f(3.14); // T = double → std::is_integral<double>::value == false
// Compiler keeps only the "not integral" branch
```

*** Think $f(42)=\text{print}(42)$

Generated machine code for $\text{print}(42)$ contains **no trace** of the "not integral" branch.

Generated machine code for $\text{print}(3.14)$ contains **no trace** of the "integral" branch.

The compiler erases the irrelevant branch at compile time.

constexpr is a keyword in c++. That tells to evaluate the value in compile time.

See this from cppinsights.io comparing with and without "constexpr"

```
1 #include <iostream>
2 #include <type_traits>
3 using namespace std;
4
5 template <typename T>
6 void print(const T& value) {
7     if(std::is_integral<T>::value) {
8         cout << value << " is an integral type\n";
9     } else {
10         cout << value << " is NOT an integral type\n";
11     }
12 }
13
14 int main() {
15     print(42); // integral
16     print(3.14); // not integral
17     cout<<"mmm";
18 }
19
```

```
18 template<
19 void print<int>(const int & value)
20 {
21     if(std::integral_constant<bool, true>::value) {
22         std::operator<<(std::cout.operator<<(value), " is an integral type\n");
23     } else {
24         std::operator<<(std::cout.operator<<(value), " is NOT an integral type\n"
25     }
26 }
27 }
28 #endif
29
30
31 /* First instantiated from: insights.cpp:16 */
32 #ifndef INSIGHTS_USE_TEMPLATE
33 template<
34 void print<double>(const double & value)
35 {
36     if(std::integral_constant<bool, false>::value) {
37         std::operator<<(std::cout.operator<<(value), " is an integral type\n");
38     } else {
39         std::operator<<(std::cout.operator<<(value), " is NOT an integral type\n"
40     }
41 }
42 }
```

```
1 #include <iostream>
2 #include <type_traits>
3 using namespace std;
4
5 template <typename T>
6 void print(const T& value) {
7     if constexpr (std::is_integral<T>::value) {
8         cout << value << " is an integral type\n";
9     } else {
10         cout << value << " is NOT an integral type\n";
11     }
12 }
13
14 int main() {
15     print(42); // integral
16     print(3.14); // not integral
17     cout<<"mmm";
18 }
19
```

```
15
16 /* First instantiated from: insights.cpp:15 */
17 #ifndef INSIGHTS_USE_TEMPLATE
18 template<
19 void print<int>(const int & value)
20 {
21     if constexpr (true) {
22         std::operator<<(std::cout.operator<<(value), " is an integral type\n");
23     } else /* constexpr */ {
24     }
25 }
26 }
27 #endif
28
29
30 /* First instantiated from: insights.cpp:16 */
31 #ifndef INSIGHTS_USE_TEMPLATE
32 template<
33 void print<double>(const double & value)
34 {
35     if constexpr (false) {
36     } else /* constexpr */ {
37         std::operator<<(std::cout.operator<<(value), " is NOT an integral type\n"
38     }
39 }
40 }
```

Build own Concept/Custom concept:

Example 1:

```
8
9  template<typename T>
10  concept myint=is_integral_v<T>;
11  ⚡
12  template<typename T>
13  requires myint<T>
14  T add(T a,T b){
15  |   return a+b;
16  }
17
18  int main(){
19  |   int a=12;
20  |   int b=12;
21  |   auto rs=add(a,b);
22  |   cout<<rs;
23  }
```

Line- 9,10: Custom concept. Here we are setting function parameter have to integral type.

Line-13: function checks whether our parameter satisfy the concept. If one parameter is float it will give compiler error.

For int value the type trait (is_integral_v) is return true and concept is satisfied. And function template is gonna execute.

For double, float value we can use "is_floating_point<T>".

Diff way to use concepts:

```
12  template<myint T>
13  T add(T a,T b){
14  |   return a+b;
15  }
16
```

```
12  template<typename T>
13  requires myint<T>
14  T add(T a,T b){
15  |   return a+b;
16  }
```

```
template<typename T>
✓ T add(T a,T b)requires myint<T>{
|   return a+b;
| }
```

```

myint auto add(myint auto a, myint auto b){
    return a+b;
}

```

Example 2:

```

9     template<typename T>
10     concept mymulti=requires(T a, T b){
11         a*b; // check if a,b is multiplyable
12     };
13
14     template<typename T>
15     requires mymulti<T>
16     T multi(T a, T b){
17         return a*b;
18     }
19
20     int main(){
21         int a=12;
22         int b=12;
23         auto rs=multi(a,b);
24         cout<<rs;
25     }

```

Line-10,11: requires 2 parameter which are multipliable. This will not give multiply of "a" and "b" if pass (char) concepts will not satisfy. Error

Example 3: If we want "a" will be int and "b" will be double. See below. Use diff type name.

```

11     template<typename T, typename U>
12     concept mymulti=requires(T a, U b){
13         a*b; // check if a,b is multiplyable
14     };
15
16     template<typename T, typename U>
17     requires mymulti<T, U>
18     // requires myint<T>;
19     T multi(T a, U b){
20         return a*b;
21     }
22
23     int main(){
24         int a=12;
25         double b=12.6;
26         auto rs=multi(a,b);
27         cout<<rs;
28     }

```

Deep dig into 'Requires':

The requires clause can take in four kinds of requirements :

- Simple requirements
- Nested Requirements
- Compound Requirements
- Type Requirements

you can only put valid expressions involving the types/objects. requires doesn't test boolean conditions — it just checks "is this expression well-formed?" check given expression is valid or not.

Inside requires { ... }, each line is an expression requirement.

It only checks whether the expression is valid (well-formed) — not whether its value is true or false.

- Use `requires { expr; }` → when you just want to check if `expr` *compiles*.
- Use `concept = condition;` → when you want to enforce a *true/false property*.

```
8  template<typename T, typename U>
9  concept siz=requires(T a, U b){
10     sizeof(T) <= 4;
11     sizeof(U) <= 4;
12 };
13 template<typename T, typename U>
14 requires siz<T, U>
15 T add(T a, U b){
16     return a+b;
17 }
18
19 int main(){
20     int a=12;
21     double b=12.6;
22     auto rs=add(a,b);
23     cout<<rs;
24 }
```

Here "sizeof(T)<=4" is simple requirement. Only check syntax.

Although b is double. It should give compiler error cause concepts does not meet(double size 8byte). It gives ans. Cause:

sizeof(T) <= 4 is always a valid expression (it produces a true value).

The compiler says "requirement satisfied" regardless of whether the condition is true or false.

It doesn't enforce the size check.

```

8   template<typename T, typename U>
9   concept siz=(sizeof(T) ≤ 4 && sizeof(U) ≤ 4);
10  ↵
11  template<typename T, typename U>
12  requires siz<T, U>
13  T add(T a, U b){
14      return a+b;
15  }
16
17  int main(){
18      int a=12;
19      double b=12.6;
20      auto rs=add(a,b);
21      cout<<rs;
22  }

```

This is correct way to set sizeof(). This will check conditions are true or false. And this will give error cause 'b' size is 8 byte. We set <=4.

Or we can use nested requirement(requires inside requires). See below(No error)

```

template<typename T, typename U>
concept siz=requires(T a, U b){
    requires sizeof(T) ≤ 4;
    requires sizeof(U) ≤ 8;
};

template<typename T, typename U>
requires siz<T, U>
T add(T a, U b){
    return a+b;
}

int main(){
    int a=12;
    double b=12.6;
    auto rs=add(a,b);
    cout<<rs;
}

```

Example *: Using Logical operator...

```
template<typename T>
requires is_integral_v<T> || is_floating_point_v<T>
T add (T a, T b){
    return a+b;
}

int main(){
    double a=12;
    double b=12.6;
    auto rs=add(a,b);
    cout<<rs;
}
```

Example **:

```
template<typename T>
concept siz=requires(T a){
    requires sizeof(T) ≤ 8;
};

template<typename T>
requires is_integral_v<T> || siz<T>
T add (T a, T b){
    return a+b;
}

int main(){
    double a=12;
    double b=12.6;
    auto rs=add(a,b);
    cout<<rs;
}
```

Exit(): "exit(1)" terminate the whole program. So if this is also used in used in a function, it stop the entire program.

exit(1) = "Stop the program right now — something went wrong."

exit(0) = "Stop the program — everything went fine."

- `exit()` doesn't "return" like a normal function.
- It calls the **system's termination process**:
 - flushes all output buffers,
 - closes all open files,
 - releases system resources,
 - and then ends the entire program.

```
exit(0);
```

it means:

→ **Immediately terminate the entire program**, just like `exit(1)`, but the difference is in the **status code** it sends to the operating system.

`exit(0)` → "Program ended successfully."

`exit(1)` → "Program ended due to an error." // abnormal termination of the program

*** If we pass an array to a function it pass its reference through pointer. Not a copy.

OOP

Class: Class is a mechanism to build our own type to use them like we have been using built in type(int, double). Its like a blueprint to create object.

Object: An object is a real instance(copy) of that class. Like a actual car built from blueprint.

```

8      class mycls{
9          public:
10             int a;
11             double b;
12             int area(){
13                 return a*b;
14             }
15     };
16
17     int main(){
18         mycls ob1;
19         ob1.a=12;
20         ob1.b=12.3;
21         cout<<ob1.a<<" " <<ob1.area();
22     }
```


* Public means after "public" all parameter are accessible outside of the class.

* If public is not defined. Then in general all parameter are indicated to private. Means can't access outside of the class. (Members of class are private by default).

* Public, private, protected are called access specifier.

* if public is not defined on above example. All member will be private. Line 19,20, 21 can't write. Compiler will give error.

* Private members can be accessible from inside class.

* Objects(ob1) are run time data.

* Member variable should be set to private.

- Class member variables can either be raw stack variables or pointers
- Members can't be references
- Classes have functions (methods) that let them do things
- Class methods have access to the member variables, regardless of whether they are public or private
- Private members of classes (variables and functions) aren't accessible from the outside of the class definition

1. Constructor:

Class constructor

A special kind of method that is called when an instance of a class is created

- No return type
- Same name as the class
- Can have parameters. Can also have an empty parameter list
- Usually used to initialize member variables of a class

Constructor are 2 type.

1. Default constructor. //Method have no parameter line: 13

2. Parameterized constructor. //Have parameter Line: 17

```

8   class mycls{
9       private:
10          int a;
11          double b;
12      public:
13          mycls(){
14              a=1;
15              b=12;
16          }
17          mycls(int x, double y){
18              a=x;
19              b=y;
20          }
21          double area(){
22              return a*b;
23          }
24      };

```

* This is how private members can be accessible inside class.

* If an object is declare default constructor will be called. Line: 13

* if not defined(default constructor), compiler will automatically generate default empty constructor.

Or inside main

"mycls ob1(12,22);" parameterized constructor will be called. Line: 17. default will not be called.

* If compiler sees any constructor, its not gonna generate default constructor.

* We can declare default constructor 2 ways; on line:13 & 16

```

9       private:
10          int a;
11          double b;
12      public:
13          mycls(){
14
15          }
16      ⚡ mycls()= default;

```

2. Setter & Getter:

Private members are not accessible from outside. Both are methods to modify or read member variable of a class.

ifndef:

```
1 #ifndef CONSTANTS_H
2 #define CONSTANTS_H
3
4 const double PI {3.1415926535897932384626433832795};
5
6 #endif
```

Means this constant is define on any other file of the project and we are not sure, we can use this. It says, if below code is not defined, I am defining here, else skip. Otherwise, if same things defined in many places, show error.

“::” Scope Resolution Operator: Used when a function of a class us defined on other file.

```
main.cpp  cylinder.h  cylinder.cpp x  constants.h
cylinder.cpp > Cylinder::Cylinder(double, double)
1 #include "cylinder.h"
2
3 Cylinder::Cylinder(double rad_param, double height_param){
4     base_radius = rad_param;
5     height = height_param;
6 }
```

Here “Cylinder” constructor is belong to Cylinder class. We can use any function instead constructor. But we have to mention function prototype inside the class. Like: **Cylinder(double red.....);**

Manage class object by Pointer:

```
#define NL printf("\n")
#define Pi 3.1416
//g++ proj.cpp && ./a.out

class circleArea{
private:
    double radius;

public:
    void set_radius(double radius){
        this->radius=radius;
    }
    double get_radius(){
        return radius;
    }
    circleArea(double radius){
        set_radius(radius);
        cout<<Pi*radius*radius;
    }
    double findcir(){
        double x=Pi*radius*radius;
        return x;
    }
};

int main(){
    circleArea c1(12);NL; // obj created on stack
    circleArea* c2= new circleArea(12);NL; // obj created on heap
    cout<<c2->findcir();
}
```

Here “->” is a dereferencing operator. Works like **(*c2).findcir()**;

c2 pointer will be on stack memory but obj will be on heap.

Direct creating object creates on stack and with using pointer object will create on heap. Remember to release memory.

```
delete c2;
```

👉 C++ will:

1. Call the **destructor** of `circleArea` (if defined)
2. Free the **entire heap memory of the object**
3. The pointer `c2` itself still exists, but now becomes **dangling**

dangling pointer is dangerous. Holds old memory. So after deleting set **c2=nullptr**;

3. Destructors:

Special methods that are called when an object dies. They are needed when the object needs to release some dynamic memory, or for some other kind of clean up.

Destructors are called by the compiler to destroy our objects.

```

#include "dog.h"

Dog::Dog(){
    dog_name = "None";
    dog_breed = "None";
    dog_age = new int;
    *dog_age = 0;
}

Dog::Dog(std::string name_param, std::string breed_param, int age_param)
{
    dog_name = name_param;
    dog_breed = breed_param;
    dog_age = new int; // Memory allocated on the heap
    *dog_age = age_param;
}

Dog::~Dog()
{
    delete dog_age;
    std::cout << "Dog destructor called for " << dog_name << std::endl;
}

```

Dynamic memory allocation

Release the memory

When Destructor Called?

- * When local stack objects goes out of scope then destructor is going to be called by compiler.
 - * When heap object is released with delete.
 - *** Destructor has no parameter.
 - *** Destructor does not called by compiler automatically for heap data. We have to release memory like line 32(below)
 - *** The compiler calls destructor for objects with automatic storage duration when they leave scope.
- If I have 3 objects(d1,d2,d3), destructor will call 3 times. But in reverse order. 'd3' destructor will call 1st and d1 destructor in last.

```

7 ~ class Dog{
8     private:
9         string Name;
10        string Breed;
11        int *Dog_age=nullptr;
12    public:
13        Dog()=default; // default constructor
14        Dog(string_view name_param, string_view breed_param, int age_param);
15        //will define later
16        ~Dog();
17 ~    string get_name(){
18        // this->Name=name;
19        return Name;
20    }
21 };
22 //class function/constructor can be defined later like below or on other file
23 ~ Dog::Dog(string_view name_param, string_view breed_param, int age_param){
24     Name=name_param;
25     Breed=breed_param;
26     Dog_age=new int; // allocation memory for age_param
27     *Dog_age=age_param;
28     cout<<"Name: "<<Name<<" Breed: "<<Breed<<" age: "<<*Dog_age;
29 }
30 //defining destructor
31 ~ Dog::~~Dog(){
32     delete Dog_age; Dog_age=nullptr;
33     cout<<"Name: "<<Name<<" Breed: "<<Breed<<endl<<"Age can't call cause deleted";
34 }
35
36 ~ int main(){
37     Dog d1("kutta", "Bangla", 10);
38     cout<<endl<<"Naaam: "<<d1.get_name()<<endl;
39     cout<<endl<<"Done!";
40
41 }

```

Explained Below:

* "d1" is a local object. It destroyed when main() exits its scope after line 39 and before the program returns from main(line 41);

* "d1" lives in stack. When main ends, C++ compiler automatically calls destructor. Memory released for 'd1' object.

string_view

is a C++17 feature (from the `<string_view>` header) that provides a lightweight, non-owning view of a string. It doesn't copy the string data—instead, it holds a pointer to the existing string data and a length, making it efficient for read-only operations.

How it works:

No ownership: It doesn't manage memory; it just "views" the string. If the original string is destroyed, the `string_view` becomes invalid (dangling reference risk).

Construction: You can create one from a `std::string`, C-string, or literals: `string_view sv = "hello";` or `string_view sv(my_string);`.

Usage: It's like a `const char*` with bounds checking. You can access characters, get substrings, compare, etc., but can't modify the data.

Efficiency: Great for function parameters when you want to avoid copying large strings. For example, in your `Dog` constructor, `string_view` lets you pass strings without copying them initially.

In line 23 *string_view*

Here, `string_view` avoids an extra copy during parameter passing, but then you copy into `Name` and `Breed` (which are `std::string`). If you want to avoid copying altogether, you could store `string_view` members instead—but be careful with lifetimes.

For more details, check the C++ reference. If you have questions about using it differently in your code, let me know!


```
cpp
void greet(std::string_view name) {
    std::cout << "Hello, " << name << "\n";
}

std::string s = "Alice";
greet(s);           // ✓ works - views s's data
greet("Bob");       // ✓ works - no allocation!
greet(s.substr(0, 3)); // ✓ works - views a slice
```

With `const string&`, calling `greet("Bob")` would silently allocate a temporary `std::string` on the heap.

The Mental Model

`string_view` is like a **window** into string data — it can look at any contiguous sequence of characters regardless of where they live (`std::string`, `char[]`, literal, etc.), without copying or owning them.

Rule of thumb: Prefer `string_view` for read-only string parameters in modern C++ (C++17+) — it's more flexible and often faster than `const string&`.

Diff Bet string_view & string:

```
C++
void set_name(const std::string_view name) {}
void set_name(const std::string name) {}
```

look similar, but they have **very different semantics and costs**. Let's analyze precisely.

1. Core difference: owning vs non-owning

- `std::string`
 - **Owning type**
 - Allocates memory and **stores its own copy**
 - Lifetime is independent
- `std::string_view`
 - **Non-owning view**
 - Just:
 - pointer (`char*`)
 - length
 - **Does NOT copy data**
 - Depends on someone else's memory

2. What happens when you pass arguments

Case A: Passing "Kutta" (string literal)

👉 `std::string_view`

⌞ C++

```
set_name("Kutta");
```

- No allocation
- Just creates a view to the literal
- Very fast

👉 `std::string`

⌞ C++

```
set_name("Kutta");
```

- Creates a temporary `std::string`
- Allocates memory
- Copies "Kutta"

➡ More expensive

Case B: Passing an existing `std::string`

⌞ C++

```
std::string s = "Kutta";
```

👉 `string_view`

⌞ C++

```
set_name(s);
```

- No copy
- Just references `s`'s buffer

👉 `string`

⌞ C++

```
set_name(s);
```

- Makes a full copy of `s`

Pass an object by value: see 22:00:00 hr

4. “this” Pointer: ‘this’ is a special kind of pointer, maintain by c++ to manipulate the current object. This ‘this’ pointer contains the address of the current object.

```
proj.cpp

Dog::Dog(){
    Name="None";
    Breed="None";
    Dog_age=new int; // allocation memory for age_param
    *Dog_age=0;
    cout<<"Name: "<<Name<<" Breed: "<<Breed<<" age: "<<*Dog_age<<this;
    //"this" will print current obj address
}
```

//“this” will print current object address

```
proj.cpp

Dog::Dog( string Name){
    this->Name=Name; // 'this' points to the current object member
}
```

Not exactly pointing to the current object member(above).

Chained call using Pointer:

```
proj.cpp

class Dog{
private:
    string Name;
    string Breed;
    int Dog_age;
public:
    Dog()=default; // default constructor
    Dog* set_name(string_view name){
        Name=name;
        return this;
    }
    Dog* set_breed(string_view breed){
        Breed=breed;
        return this;
    }
    Dog* set_age(int age){
        Dog_age= age;
        return this;
    }
    void printInfo(){
        cout<<Name<<" "<<Breed<<" Age: "<<Dog_age<<" address at: "<<this;
    }
};

int main(){
    Dog* d1=new Dog();
    d1->set_name("Kutta")->set_breed("Bangla")->set_age(5);
    d1->printInfo();
    cout<<endl<<"Done!";
}
```

Chained call using Reference:

```
7 class Dog{
8     private:
9         string Name;
10        string Breed;
11        int Dog_age;
12    public:
13        Dog()=default; // default constructor
14        Dog& set_name(const string_view& name){
15            Name=name;
16            return *this; // return reference of Dog object
17        }
18        Dog& set_breed(const string_view& breed){
19            Breed=breed;
20            return *this;
21        }
22        Dog& set_age(int age){
23            Dog_age= age;
24            return *this;
25        }
26        void printInfo(){
27            cout<<Name<<" "<<Breed<<" Age: "<<Dog_age<<" " address at: "<<this;
28        }
29    };
30
31    int main(){
32        Dog* d1=new Dog();
33        d1->set_name("Kutta").set_breed("Bangla").set_age(5);
34        d1->printInfo();
35        delete d1;
36        cout<<endl<<"Done!";
37    }
```

Here 'const' is important to include.

What happen in line: 33

Here's what happens:

1. `d1->set_name("Kutta")` - Sets the dog's name to "Kutta" and returns a reference to the Dog object (`Dog&`)
2. `.set_breed("Bangla")` - Called on the returned reference, sets the breed and returns another reference to the same Dog object
3. `.set_age(5)` - Called on the returned reference, sets the age

This works because each setter method (`set_name`, `set_breed`, `set_age`) returns `Dog&` (a reference to `*this`), allowing you to chain multiple calls together in a single statement. It's a common pattern called the "fluent interface" or "builder pattern".

5. Struct Vs Classes: Struct is another way to define classes. Differences:

```
</> C++

struct A {
    int x;    // default: public
};

class B {
    int x;    // default: private
};
```

- **struct** → members are **public** by default
- **class** → members are **private** by default

- **struct** → inherits **publicly** by default
- **class** → inherits **privately** by default

So We can use struct when we have public members. But both are almost same. We can use anyone.

6. Size of the class objects: Size of the class objects depends on the member of the class, not functions.

```
13 };
14
15 int main(){
16     cout<<"1: int "<<sizeof(int)<<endl;
17     cout<<"1: int* "<<sizeof(int*)<<endl;
18     string s="";
19     cout<<"1: string "<<sizeof(s)<<endl;
20 }
```

```
• emon fedora ~/code
g++ proj.cpp && ./a.out
1: int 4
1: int* 8
1: string 32
```

Here, string size is 32 byte whether I put no character or put millions of character. It will remain same. We are not getting memory which occupy by characters, but getting the fixed size of the string object itself in the memory. It includes internal pointers, metadata(like length, capacity) and other housekeeping data.

If I put millions of characters inside string, it will store in separate dynamically allocated memory in the heap. Above is in general. But for small string, there may be diff scenario: below

```
14
15 int main(){
16     // Test with 1 character
17     string one_char = "a";
18     cout << "1 char string sizeof: " << sizeof(one_char) << " bytes" << endl;
19     cout << "1 char string size(): " << one_char.size() << endl;
20     cout << "1 char string capacity(): " << one_char.capacity() << endl;
21
22     // Test with 16 characters (likely on heap)
23     string sixteen_chars = "aaaaaaaaaaaaaaaa";
24     cout << "16 char string sizeof: " << sizeof(sixteen_chars) << " bytes" << endl;
25     cout << "16 char string size(): " << sixteen_chars.size() << endl;
26     cout << "16 char string capacity(): " << sixteen_chars.capacity() << endl;
27 }
```

```
• emon fedora ~/code g++ pr
oj.cpp && ./a.out
1 char string sizeof: 32 bytes
1 char string size(): 1
1 char string capacity(): 15
16 char string sizeof: 32 bytes
16 char string size(): 16
16 char string capacity(): 16
```


so if i put 1 character it will on the heap memory, not inside 32 byte string object memory?

✓ Reviewed and updated proj.cpp

✓ `g++ proj.cpp && ./a.out`

Yes, that's a great question! The answer depends on the string's length and the C++ implementation. Most modern compilers (like g++) use **Small String Optimization (SSO)** for short strings.

What Happens with 1 Character?

- For very short strings (typically up to 15 characters in g++), the data is often stored **directly inside the 32-byte string object** (in a small internal buffer) to avoid the overhead of heap allocation.
- This is SSO in action—it makes short strings faster and avoids dynamic memory for common cases.

What Happens with Longer Strings?

- Once the string exceeds the SSO threshold (around 15-16 characters), the data moves to **heap memory**, and the string object just holds pointers to it.

From the output of the modified code:

- 1 character: `capacity()` is 15 (indicating the internal buffer in the object is being used).
- 16 characters: `capacity()` is 16 (indicating heap allocation).

The `sizeof()` is always 32 bytes (the object size), but the actual storage location of the characters changes based on length. For performance, short strings stay "inside" the object, while long ones go to the heap. This is why strings are efficient for both small and large data.

🔄 ↶ 🌟 🔖

Grok Code Fast 1 • 1x

Why `int` is 4 and `int*` is 8

- `sizeof(int)` gives the size of the integer value itself. On your system, `int` is 4 bytes.
- `sizeof(int*)` gives the size of the pointer, not the integer. A pointer stores a memory address.

On your machine

- You are on a 64-bit system.
- A memory address is 8 bytes on 64-bit systems, so `int*` is 8 bytes.
- The pointer size is independent of the type it points to: `char*`, `double*`, `int*` are usually all 8 bytes.

Summary

- `int` = 4 bytes because that is the size of the integer value.
- `int*` = 8 bytes because it stores a 64-bit address pointing to an `int`.

Object size:

```
7 class human{
8     private:
9         int age; // 4 byte
10        string Breed; //32 byte
11        double Dog_age; //8 byte
12    };
```

Here object size should be 44 byte. But Actual size is 48. cause, Alignment rule

Each type has an **alignment requirement**:

- `std::string` → usually 8-byte aligned
- `double` → 8-byte aligned
- `int` → 4-byte aligned

The total object size must be a multiple of the largest alignment (here: 8 bytes)
so there will 4 byte padding after 4 byte integer memory. See memory layout: below

Offset	Size	Member
0	4	age
4	4	padding (to align next member to 8)
8	32	Breed
40	8	Dog_age

Here Offset is the byte position of a member inside an object, measured from the object start.
For functions:

Why function parameters are not part of object size

- `sizeof(human)` counts only the memory needed for the object's actual data members.
- A function parameter is not a data member; it is temporary storage used only when the function runs.

Key points

- Member functions are stored once as code, not duplicated per object.
- When you call a function, its parameters live in the call frame:
 - usually in CPU registers
 - or on the stack
- That memory is separate from the object itself.

Inheritance

1. Inheritance: Inheritance in C++ is a mechanism that allows a new class (derived class) to inherit properties and behaviours (methods) from an existing class (base class), promoting code reuse and creating a hierarchical relationship between classes.

When a class inherits another class, it gets all the accessible members of the parent class, and the child class can also redefine (override) or add new functionality to them.

- With public inheritance , derived classes can access and use public members of the base class, but the derived class can't directly access private members
- The same also applies to friends of the derived class. They have access to private members of derived, but don't have access to the base class

Public, private, protected are access specifier.

```

6
7 class human{
8     private:
9         string firstName;
10        string lastName;
11    public:
12        int age;
13 };
14 class player : public human{
15     public:
16     player(string_view firstName, string_view lastName, int age){
17         // this->firstName=firstName; // private member. can't inherit
18         // this->lastName=lastName; // private member. can't inherit
19         this->age=age;
20     }
21 };
22
23 int main(){
24     player manus("Motaher", "emon", 150);
25     cout<<manus.age; //public member. player class can access
26     // cout<<manus.firstName; //private member. player class cant access
27 }

```

in the above example, player class inherits all public members of human. But can't access private. That's why we only can print 'age' variable. Also passing string through player object("motaher", "emon") constructor will be unused. We have to use public setter & getter.

***Protected member: Sometime we may want that members of base class has to be accessible from derived class but still be inaccessible from outside. In that case use those member as protected member. Below:

```

6
7 class human{
8     protected:
9         string firstName;
10        string lastName;
11    public:
12        int age;
13 };
14 class player : public human{
15     public:
16     player(string_view fName, string_view lName, int age){
17         this->firstName=fName; // protected member. can inherit
18         this->lastName=lName; // protected member. can inherit
19         this->age=age;
20     }
21     void print(){
22         cout<<"FirstName: "<<firstName<<" "<<"LastName: "<<lastName<<NL;
23     }
24 };
25
26 int main(){
27     player manus("Motaher", "emon", 150);
28     cout<<manus.age<<NL; //public member. player class can access
29     // cout<<manus.firstName; //protected member. can't access from outside
30     manus.print();
31 }

```

Terminal output:

```

g++ proj.cpp &
./a.out
150
FirstName: Motaher LastName: emon

```

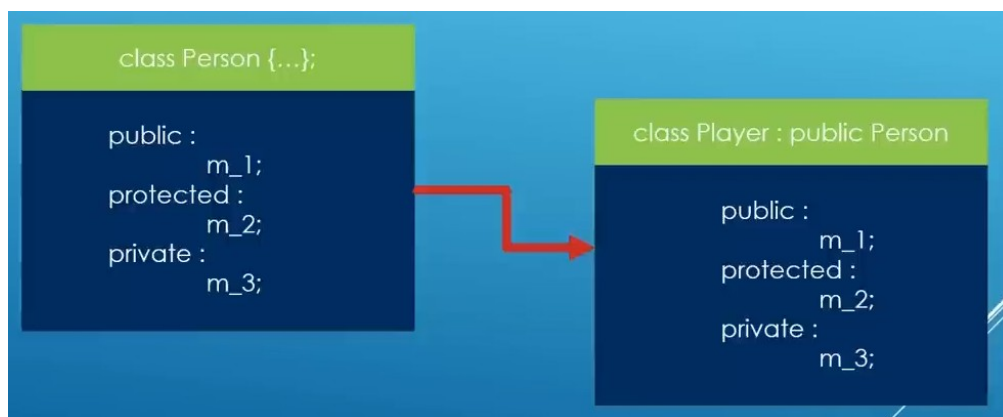
***Base class access specifier:

```

14 class player : public human{

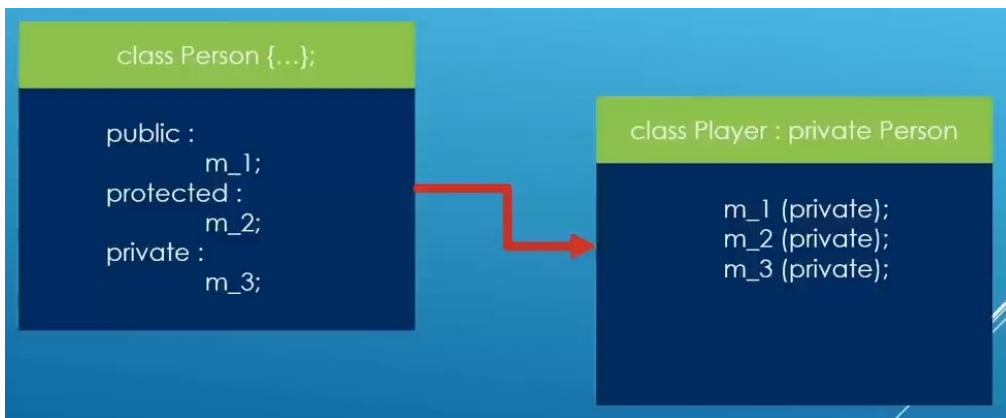
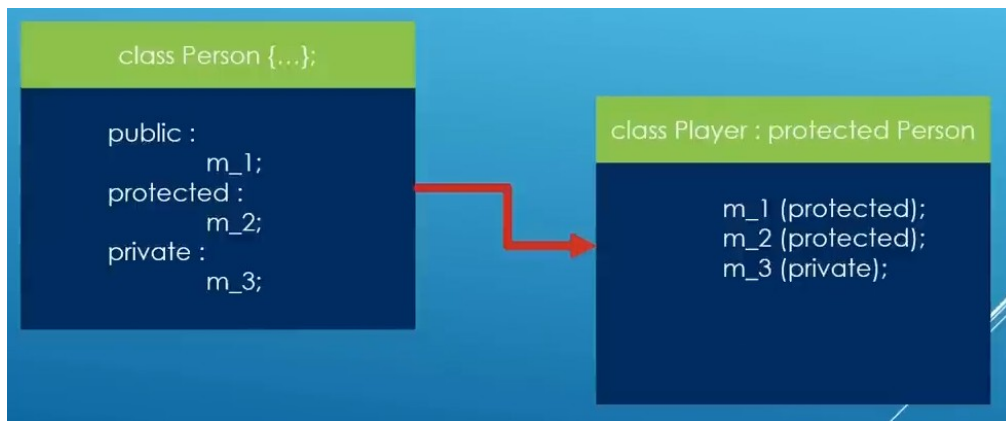
```

Here "Public" is base class access specifier. This is public inheritance. This can be private/protected. This tells how accessible the base class members are to the derived class.



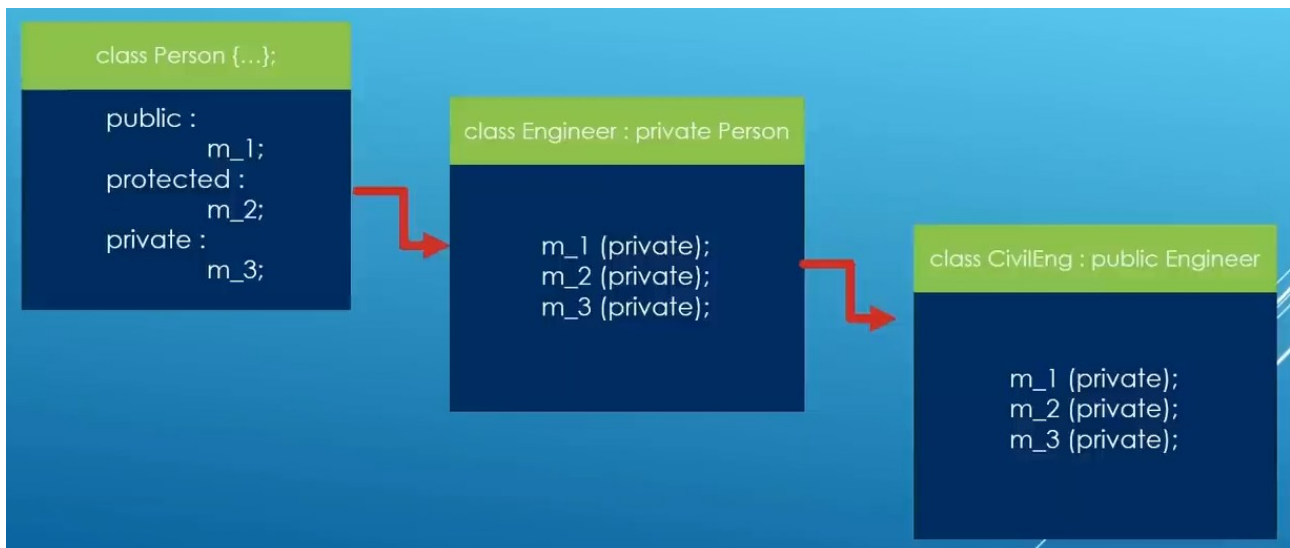
Public Inheritance:

1. Anything public in base class, will remain public in derived class.
 2. Anything protected in base class, will remain protected in derived class.
 3. Private members are never inherited. Derived class can't access.
- Others (private, protected) we can derive from our assumption.



Access Table			
Base Member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	✗ inaccessible	✗ inaccessible	✗ inaccessible

Private members are never inherited — regardless of inheritance type.



Here, Engineer class is inherited by CivilEngr class. And Person class is inherited by Engineer class privately. So CivilEngr class can't access any member from person class. Cause all members(except m_3) from base class(person) is private to Engineer class. Engineer class can access. But can't CivilEngr..

But But But

C++ allow us do something else so we can access from civilEng class.

we can use m_1, m_2 member in CivilClass by using "using" keyword. See below:

```
7  class person{  
8      public:  
9          string pName;  
10         string pFatherName;  
11         person()=default;  
12         person(string_view pName, string_view pfatherName, string_view pmName, int age){  
13             this->pMotherName=pmName;  
14             this->pAge=age;  
15             this->pName=pName;  
16             this->pFatherName=pfatherName;  
17         }  
18         string get_pMotherName(){  
19             return pMotherName;  
20         }  
21         int get_Age(){  
22             return pAge;  
23         }  
24  
25     protected:  
26         string pMotherName;  
27  
28     private:  
29         int pAge;  
30 };
```

```

31
32 class manush: private person{
33     //all member will be private here.
34     //A class which will inherit this class , can't access any member, unless using "using" as below
35     public:
36         manush()=default;
37         using person::pMotherName;
38         //now "pMotherName" member will be accessible to that class which will inherit this class
39
40 };
41 class myFamily: private manush{
42     //this class only access "pMotherName" member
43     //cause we made this member public in manush class.
44     public:
45         myFamily(string_view pmotherName){
46             this->pMotherName=pmotherName;
47         }
48         string getmotherName(){
49             return pMotherName;
50         }
51 };

```

```

52
53 int main(){
54     person ami("Emon", "Abbu", "Amma", 32);
55     cout<<ami.pName<<endl<<ami.pFatherName<<endl<<ami.get_pMotherName()<<endl<<ami.get_Age();
56     NL;
57     myFamily amii("Amr amma");
58     cout<<amii.getmotherName();
59 }

```

Here 'using' does is Bring member 'pMotherName' from class A into current class scope. It re-expose the member.

Though "pMotherName" member is protected. We can use in myFamily class, explained below:

- `person::pMotherName` is protected, so it is normally accessible to derived classes.
- But `manush` inherits `person` as private.
- With private inheritance, `person` members become private inside `manush`, so `myFamily` cannot see `pMotherName` unless `manush` re-exposes it.

Fix step 1: expose `person::pMotherName` in `manush`

Add a `using` declaration inside `manush`:

```

class manush : private person {
protected:
    using person::pMotherName;

```

2. Default Arc Constructor with inheritance:

Always provide default constructor for classes. Because compiler may calls these default constructor, specially if the class of inheritance.

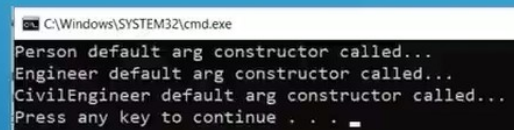

```

#include <iostream>
#include "person.h"
#include "engineer.h"
#include "civilengineer.h"

int main(int argc, char **argv)
{
    CivilEngineer civil_eng1;

    return 0;
}

```



```

C:\Windows\SYSTEM32\cmd.exe
Person default arg constructor called...
Engineer default arg constructor called...
CivilEngineer default arg constructor called...
Press any key to continue . . .

```

If we build a civilengr object and we don't have a default constructor for person class, compiler is going to call that, but will not find and give error.

Compiler will go for base class first then the other layer which was inherited to call all the constructor.

3. Initializer List:

Give a variable/object its first valid value at the moment it is created.

1. Initialization

```

<> C++
int x = 5;

```

Meaning:

create x with value 5

2. Assignment

```

<> C++
int x;
x = 5;

```

Meaning:

create x first
then later change value to 5

In constructor initializer list

```
«» C++  
  
Person(string n) : name(n) {  
}
```

Means:

```
create member `name`  
directly using `n`  
BEFORE constructor body starts
```

Without initializer list:

```
«» C++  
  
Person(string n){  
    name = n;  
}
```

Means:

1. `name` first initialized automatically
2. then reassigned inside constructor

```
7  class Car{  
8      public:  
9          string CarName;  
10         int weight;  
11         Car(string s, int n): CarName(s), weight(n){} //with Initializer List  
12     };  
13     class Car1{  
14         public:  
15             string CarName;  
16             int weight;  
17             Car1(string s, int n){ //without Initializer List  
18                 this->CarName=s;  
19                 this->weight=n;  
20             }  
21     };  
22 
```

Explain: Initializer List vs Assignment

```
1. class Car uses an initializer list:
Car(string s, int n): CarName(s), weight(n) {}

2. class Car1 uses assignment inside the constructor body:
Car1(string s, int n) {
    this->CarName = s;
    this->weight = n;
}
```

What changes

- class Car constructs CarName and weight directly with the values s and n.
- class Car1 first default-constructs CarName and weight, then assigns new values to them inside the constructor body.

Which is better and why

class Car with the initializer list is better.

Reasons:

- More efficient: member variables are initialized once instead of default-initialized then assigned.
- Required for certain members: const, reference members, or members without default constructors must be initialized in the initializer list.
- Clearer intent: it shows initialization order explicitly.

Practical Impact here

For string CarName and int weight, both versions work, but class Car is still preferable for style and performance.

So the initializer-list version is the better choice.

4. Custom constructor with Inheritance:

Some time we need to call custom constructor to be called instead of default with many parameter.

```
7 class vehicle{
8 public:
9     string VName;
10    int weight;
11    vehicle(string_view vname, int weightt, string_view brname)
12        :VName(vname), weight(weightt), brandName(brname){
13        cout<<VName<<" "<<weight<<" "<<brandName<<endl;
14    };
15 private:
16     string brandName;
17 };
18 class car:private vehicle{
19 public:
20     int doorNumber;
21     car(string_view vname, int weightt, string_view brName, int drnumber)
22         :vehicle(vname, weightt, brName), doorNumber(drnumber){
23         cout<<VName<<" "<<weight<<" BrandName: "<<brName<<" doorNumber: "<<doorNumbe:
24     }
25 };
26
27 int main(){
28     car("gari", 320, "Audi", 4);
29 }
```

Terminal output:

```
gari 320 Audi
gari 320 BrandName: Audi doorNumber: 4
```

The car class constructor uses an initializer list to initialize both the base class and its own members.
Initializer List: :vehicle(vname, weight, brName), doorNumber(drnumber)

1. `:vehicle(...)` - Calls the base class `vehicle` constructor, passing the name, weight, and brand name to initialize inherited members
2. `doorNumber(dnumber)` - Initializes the `doorNumber` member variable with the provided value

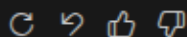
Order of initialization:

- Base class members are initialized first (via the base constructor call)
- Derived class members are initialized next (in order of their declaration)

Why use initializer lists?

- **More efficient** - Initializes members directly rather than default-constructing then assigning
- **Required for** - `const` members, reference members, and base classes
- **Cleaner syntax** - Separates initialization from other logic in the constructor body

In this case, the initializer list ensures that when a `car` object is created, both the inherited `vehicle` properties and the car-specific `doorNumber` are properly initialized before any code in the constructor body executes.



Claude Haiku 4.5 • 1x

5. Copy Constructors with Inheritance: A copy constructor in C++ is a member function that initializes a new object using an existing object of the same class. It is the standard mechanism for duplicating object data and takes a reference to the source object as its parameter.

```
35 | player manus("Motaher", "emon", 150);  
36 | player manus1(manus);
```

Here a new object initialize using existing object(`manus`).

```

7  class human{
8      protected:
9          string firstName;
10         string lastName;
11     public:
12         int age;
13         //constructor
14         human(string_view FirstName,string_view LastName, int Age)
15             :firstName(FirstName), lastName(LastName), age(Age){
16             cout<<"New Object is created with human class parameter";NL;
17         }
18
19         //copy constructor
20         human(const human& source) // both (){} works
21             :firstName(source.firstName), lastName{source.lastName}, age(source.age){
22             cout<<"Base class copy constructor Called.";NL;
23         }
24     };
25     class player : public human{
26     public:
27         int playing_age;
28         player(string_view FirstName,string_view LastName, int Age, int playingAge)
29             :human(FirstName, LastName,Age), playing_age(playingAge){
30             cout<<"New Object is created with player class parameter";NL;
31         }
32
33         //copy constructor
34         player(const player& source):human(source), playing_age(source.playing_age){
35             cout<<"Derived class copy constructor Called.";NL;
36         }
37     };
38
39     int main(){
40         player manus("Motaher", "emon", 150, 20);
41         player manus1(manus);
42     }

```

Here, in Line 20: copy constructor of human class is receiving an object reference and initialize the member value from the source.

~~~

human(source); //line 34

~~~

this means: copy the human portion from the source using human copy constructor(line 34)

~~~

human(const human& source); //line 20 or 34 same

~~~

This Means: Take a human object by reference.

Flow chart for above code:

- player "manus" object is created.
- player copy constructor called for "manus1" and passing "manus" object as as constant reference
- human(source) executed

- human copy constructor received a object reference of “manus” and copy its part and initialize “manus1” members of human class.
- Back to the player copy constructor. And copy and initialize player class member for “manus1”.

3. Why copy constructor is needed

Because C++ must answer this question:

“How exactly should a new object be built from an existing one?”

That rule is defined by the **copy constructor**.

Without it, C++ would not know:

- how to copy `string`
- how to copy base class part
- how to copy dynamic memory (if any)
- whether deep copy or shallow copy is needed

So copy constructor defines:

“Construction logic for cloning an object”

When do we need:

3. When storing objects in containers

Containers often store copies.

```
</> C++  
vector<player> v;  
v.push_back(manus); // copy or move happens
```

You want to keep the original unchanged.

```
</> C++  
player original("A", "B", 10, 5);  
player backup = original;
```

Now:

- `original` stays unchanged
- `backup` can be modified freely

Very common in real software:

- game states
- editor undo/redo
- database transaction rollback

```
</> C++  
history.push_back(currentState);
```

Each entry must be an independent copy.

Core rule (very important)

*“Copy constructor is NOT for normal use — it is for **creating a new independent object with same state when the program explicitly needs duplication.**”*

6. Inheriting Base Constructor: // introduced in C++11

Base constructors are not inherited by default. Means it is not possible to use base constructor by derived class. But it is possible to tell compiler to use the base constructor to set up our own objects. How? See below:

```
25 class player : public human{
26     using human::human; //Inheriting the constructor
27     public:
28         int playing_age;
29 };
```

line 26 says: when building player object,, don't use your own constructors instead set up a base like constructor which is going to only initialize the base member variable. And compiler is going to generate a constructor that may looks like below:

```
player(const string& fname, const string& lname, const int Age)
:human(fname, lname, Age){}
```

using human::human; makes all accessible constructors of the base class (human) available in the derived class (player). The compiler generates forwarding constructors that call the corresponding base-class constructors.

***A derived class can still define its own constructors even when using `using Base::Base;`. Both inherited constructors and user-defined constructors may coexist.

*** Only accessible constructors are inherited. Private constructors of the base class cannot be inherited.

Constructor name will be derived class name. This is going to forwarding the work to initialize our object throw the player class. This is called Inheriting base constructor.

Generally it is not recommended to use unless we don't have other option.

Q: if i have default constructor and parameterized constructor, which one will inherit if use "using human::human;"

Ans: **using Human::Human;** inherits all accessible constructors from the base class, not just one.

```

class Human {
public:
    Human() {
        cout << "Default\n";
    }

    Human(string name) {
        cout << "Parameterized\n";
    }
};

class Player : public Human {
public:
    using Human::Human;
};

```

The compiler behaves roughly as if it generated:

```

</> C++

class Player : public Human {
public:
    Player() : Human() {}

    Player(string name) : Human(name) {}
};

```

Now you can do:

```

</> C++

Player p1;           // calls Human()
Player p2("Emon");   // calls Human(string)

```

using Base::Base; inherits **every accessible constructor** from the base class (default, parameterized, copy, move, etc., if applicable). If the derived class declares a constructor with the same signature, the derived-class constructor takes precedence for that signature.

7. Inheritance with Destructor: As we know, When creating a derived-class object, constructors are called from base to derived. **But** Destructor are called in reverse order. From above example, player class destructor (derived class) will call 1st, then human (base) class destructor will call. If we have 3 layer of inheritance as videos, CivilEnginne destructor called 1st, then Engineer, then Person class destructor. In C++, destructors in inheritance are mainly about **order of destruction**.


```
7 class human{
8     public:
9         string firstName;
10        string lastName;
11    public:
12        int age;
13        //constructor
14        human(string_view FirstName,string_view LastName, int Age)
15        :firstName(FirstName), lastName(LastName), age(Age){
16            cout<<"Base(HUuman) class constructor";NL;
17        }
18
19        ~human(){
20            cout<<"Destructor of Base(Human) class.";NL;
21        }
22    };
23    class player : public human{
24        public:
25            int playing_age;
26
27            player(const string& fname, const string& lname, const int Age, int p_age)
28            :human(fname, lname, Age), playing_age(p_age){
29                cout<<"Derived(player) class constructor";NL;
30            }
31            ~player(){
32                cout<<"Destructor of Derived(player) class";NL;
33            }
34        };
35    };
36
37    int main(){
38        player manus("Motaher", "emon", 150, 20);
39        cout<<"exiting main program";NL;
40    }
```

emon fedora ~/code

emon fedora ~/code

emon fedora ~/code

emon fedora ~/code g++ proj.cpp && ./a.out

Base(HUuman) class constructor
Derived(player) class constructor
exiting main program
Destructor of Derived(player) class
Destructor of Base(Human) class.

emon fedora ~/code

emon fedora ~/code

See above: Destructor's are called in reverse order.

8. Reused symbols in Inheritance: We can use same names of parameter/methods in inheritance hierarchy. Means Base class and derived class can have same name of parameter/methods. If we have same method on child and parent class and call the method, what C++ does is, it is going to override/hide the parent class method/parameter and use the child class method/parameter

```
7 class human{
8     public:
9         int age=32;
10        void printAge(){
11            cout<<"Human Age: "<<age<<NL;
12        }
13    };
14
15    class player : public human{
16        public:
17            int age=5;
18            void printAge(){
19                cout<<"Player Age: "<<age<<NL;
20            }
21    };
22
23    int main(){
24        player manus;
25        manus.printAge();//override base "printAge" method
26        manus.human::printAge(); // access base "printAge" method
27    }
```

emon fedora ~/code
roj.cpp && ./a.out
Player Age: 5
Human Age: 32

Polymorphism

Polymorphism is an OOP concepts that allows a same function or operator to behave differently in different scenarios depending on the objects that uses it. Polymorphism means "many forms".

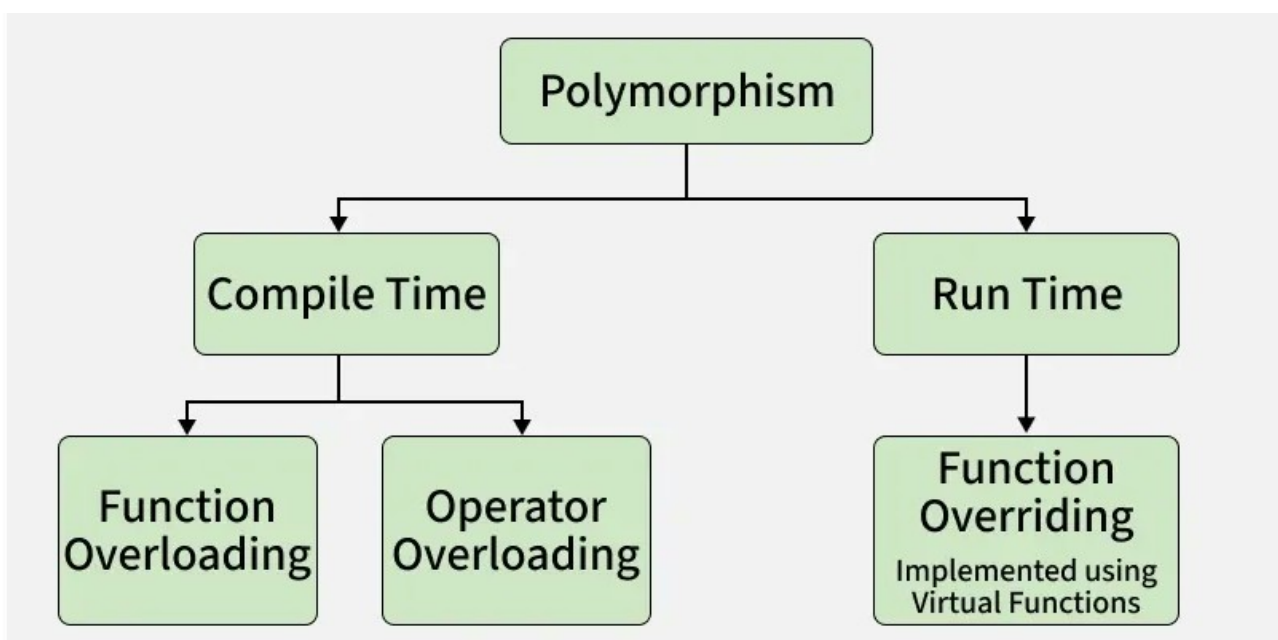
For Example;

The "+" operator in C++ is used to perform two specific functions. When it is used with numbers, it performs addition. And when we use the + operator with strings, it performs string concatenation.

Key Idea is: one interface- many implementation..

Types:

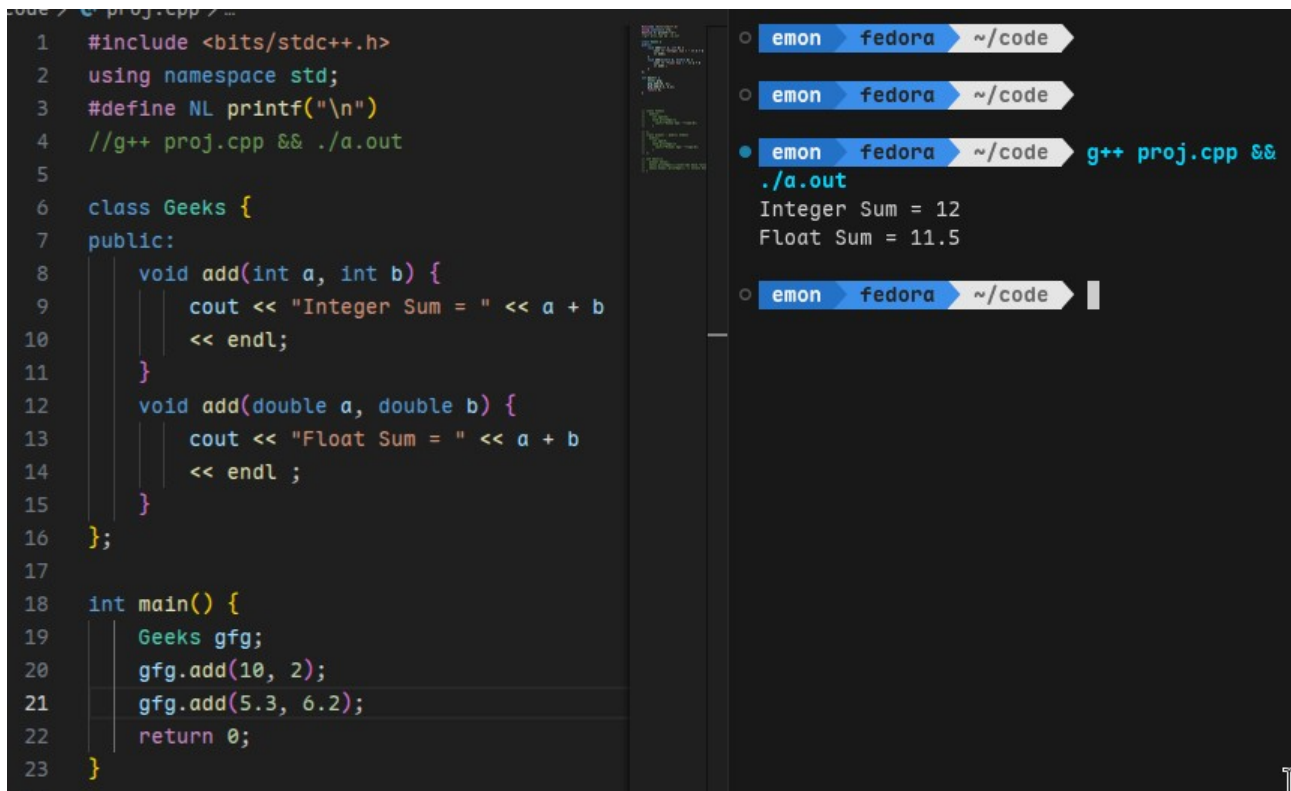
- Compile Time(Static) Polymorphism: The compiler decides which function/operator to call.
- Runtime time(Dynamic) polymorphism: The function call(which will be called)is decided while the program is running based on the object being used.



Compile Time(Static) Polymorphism: Compile-time polymorphism is also known as static polymorphism or early binding. In this type of polymorphism, the compiler decides which function or operator to call during compilation.

1. Function Overloading: In C++, we can use two functions having the same name if they have different parameters (either types or number of arguments). And, depending upon the number/type of arguments, different functions are called. For example, See the link(Click): [Example](#)

In case of OOP example:



```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define NL printf("\n")
4 //g++ proj.cpp && ./a.out
5
6 class Geeks {
7 public:
8     void add(int a, int b) {
9         cout << "Integer Sum = " << a + b
10        << endl;
11    }
12    void add(double a, double b) {
13        cout << "Float Sum = " << a + b
14        << endl ;
15    }
16 };
17
18 int main() {
19     Geeks gfg;
20     gfg.add(10, 2);
21     gfg.add(5.3, 6.2);
22     return 0;
23 }
```

emon fedora ~/code
emon fedora ~/code
● emon fedora ~/code g++ proj.cpp && ./a.out
Integer Sum = 12
Float Sum = 11.5
emon fedora ~/code

The class Geeks contains two overloaded functions named add(), one for integers and one for double values.

When add(10, 2) is called, the integer version is executed; when add(5.3, 6.2) is called, the double version is executed. The compiler decides which function to call at compile time based on the argument types, demonstrating function overloading.

2. Operator Overloading: In C++, we can define how operators behave for user-defined types like class and structures. For example,

The `+` operator, when used with values of type `int`, returns their sum. However, when used with objects of a user-defined type, it is an error.

In this case, we can define the behaviour of the `+` operator to work with objects as well.

This concept of defining operators to work with objects and structure variables is known as operator overloading.

Note: Most operators in C++ can be overloaded, but operators such as `::`, `.`, `.*`, `?.` and `sizeof` cannot be overloaded because they are essential to the core functionality of the language.

Syntax:

```
returnType operator symbol (arguments) {  
    ... ..  
}
```

Example to add 2 complex number:

Version I:

```
7  class Complex {  
8  public:  
9      int real, imag;  
10  
11      Complex(int r, int i) :  
12          real(r), imag(i) {}  
13      Complex operator+(const Complex& obj) {  
14          return Complex(real + obj.real, imag + obj.imag);  
15      }  
16  };  
17  
18  int main() {  
19      Complex c1(10, 5), c2(2, 4);  
20      Complex c3 = c1 + c2;  
21      cout << c3.real << " + i" << c3.imag;  
22      return 0;  
23  }
```

(Above code) Normally, `+` only works on built-in types like `int` or `float`. When you write `1 + 2`, the compiler knows how to add raw numbers. But `c1 + c2` where `c1` and `c2` are `Complex` objects means nothing to the compiler by default — it doesn't know what "adding" two objects should mean.

Operator overloading lets you **teach the compiler what `+` should do** when it sees two `Complex` objects on either side of it. Under the hood, `c1 + c2` literally gets translated by the compiler into a function call.

Overloading just means: "when you see this operator with these operand types, call this function instead of giving an error."

The compiler rewrites line 20 (`Complex c3 = c1 + c2;`) internally as:

```
cpp  
  
Complex c3 = c1.operator+(c2);
```

So:

- *c1* is the object the method is called **on** — that's why inside the function, *real* and *imag* (no prefix) refer to *c1*'s members.
- *c2* is the **argument** passed in — that's *obj* inside the function.
- The function returns a brand-new *Complex* built from the sums, and that becomes *c3*.

“operator+” is just a member function(of the class) (a weird-looking one) that takes one parameter, because the left-hand object is implicitly “this”.

Version II:

```
6  class Complex {
7      private:
8          float real;
9          float img;
10     public:
11         Complex() : real(0), img(0) {}
12         Complex(float real, float img) : real(real), img(img){}
13
14         friend Complex operator + (const Complex& obj1, const Complex& obj2) {
15             Complex temp;
16             temp.real = obj1.real + obj2.real;
17             temp.img = obj1.img + obj2.img;
18             return temp;
19         }
20
21         void display() {
22             if (img < 0)
23                 cout << "Output Complex number: " << real << img << "1";
24             else
25                 cout << "Output Complex number: " << real << "+" << img << "1";
26         }
27     };
28
29     int main() {
30         Complex c1(1.0f, 2.0f);
31         Complex c2(1.0f, 3.0f);
32         Complex result = c1 + c2;
33         result.display();
34
35         return 0;
36     }
```

Here, *operator+* is **not** a member of the class — it's declared friend, meaning it's a regular standalone function that's just been granted permission to access the class's private members (*real*, *img*).

The compiler rewrites line 20(*Complex c3 = c1 + c2;*) internally as:

19

cpp

```
Complex result = operator+(c1, c2);
```


Notice the difference: now **both** operands are explicit parameters, **obj1** and **obj2**. There's no implicit **"this"**, because this function isn't attached to any particular object — it's a free function that happens to live inside the class definition (for organizational convenience) and gets **friend** access to read private data.

Q: Tell me what is **Friend function**?

Ans: A **friend function** is a function that is **not a member** of a class, but is still given **special permission to access the class's private and protected members**, as if it were a member.

Sometimes we may need to access private members of a class from outside. Friend function gives us that scope.

write the function signature inside the class, with the **"friend"** keyword in front. That's just a declaration — it tells the compiler "this function, even though it's defined elsewhere (or right here), gets access to my private stuff."

```
class Complex {
private:
    int real, imag;
public:
    Complex(int r, int i) : real(r), imag(i) {}

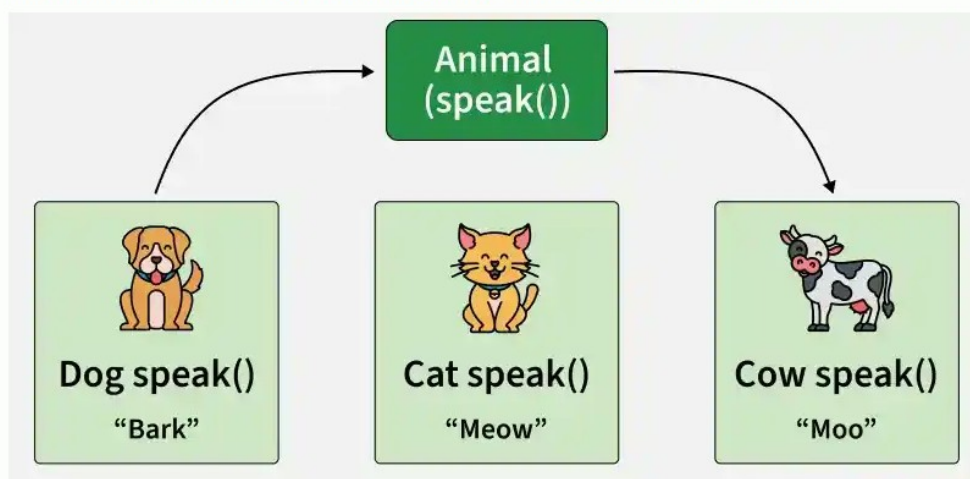
    friend void printIt(Complex c); // friend declaration
};

void printIt(Complex c) {
    cout << c.real << " " << c.imag; // now this works – friend grants access
}
```

Runtime(Dynamic) Polymorphism: Runtime polymorphism is also known as dynamic polymorphism or late binding. In this type, the function call is resolved during program execution instead of compilation.

Real-Life Illustration of Polymorphism

Different animals represent polymorphism, where the same method `speak()` produces different outputs such as Bark, Meow, and Moo depending on the object calling the method.

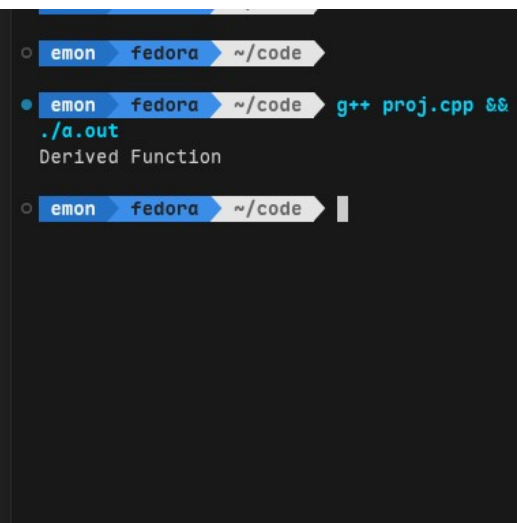


3. Function Overriding: Function Overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be overridden. The base class function must be declared as **virtual function** for runtime polymorphism to happen.

Suppose we define the same function in both the base class and the derived class. Now, when we call this function using the object of the derived class, the function of the derived class executes.

Here, the member function in the derived class shadows the member function in the base class. This is called shadowing the base class member function.

```
7  class Base {
8      public:
9          void print() {
10             cout << "Base Function" << endl;
11         }
12     };
13     class Derived : public Base {
14     public:
15         void print() {
16             cout << "Derived Function" << endl;
17         }
18     };
19     int main() {
20         Derived derived1;
21         derived1.print();
22         return 0;
23     }
```



```
emon fedora ~/code
emon fedora ~/code g++ proj.cpp &&
./a.out
Derived Function
emon fedora ~/code
```

Here, the `print()` from `Derived` is executed by shadowing the function in `Base`.

To access the shadowed function of the base class, we use the scope resolution operator `::` (below line 24)

We can also access the shadowed function by using a pointer of the base class to point to an object of the derived class and then calling the function from that pointer.

```
19     int main() {
20         Derived derived1, derived2;
21         // access print() function of the derived class
22         derived1.print();
23
24         // access print() function of the Base class
25         derived2.Base::print();
26         return 0;
27     }
```

Call the Shadowed function from derived class(below)

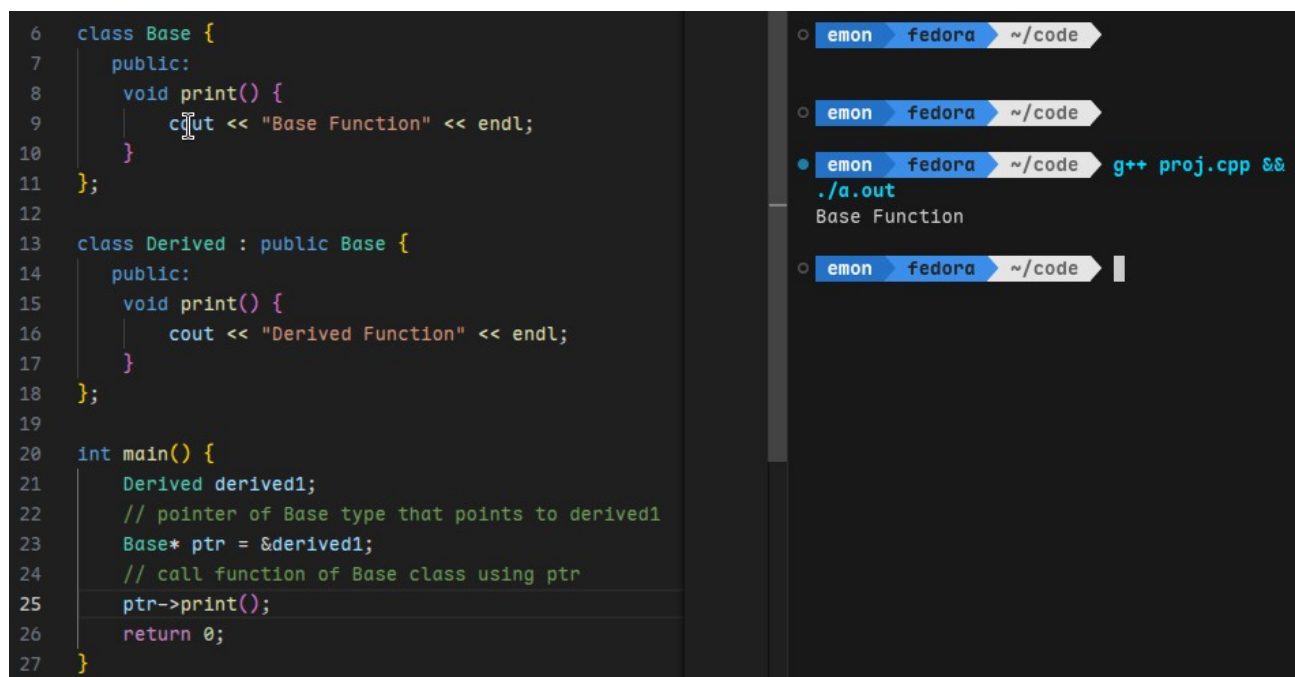
```
class Derived : public Base {
public:
    void print() {
        cout << "Derived Function" << endl;
        Base::print();
    }
};
```


Notice the code `Base::print();`, which calls the member function `print()` from the `Base` class inside the `Derived` class.(above)

Using Pointer:

(without virtual function)

**** N.B: This is not example of polymorphism. Example of Static binding**



The screenshot shows a C++ IDE with a code editor on the left and a terminal on the right. The code editor contains the following code:

```
6 class Base {
7     public:
8     void print() {
9         cout << "Base Function" << endl;
10    }
11 };
12
13 class Derived : public Base {
14     public:
15     void print() {
16         cout << "Derived Function" << endl;
17     }
18 };
19
20 int main() {
21     Derived derived1;
22     // pointer of Base type that points to derived1
23     Base* ptr = &derived1;
24     // call function of Base class using ptr
25     ptr->print();
26     return 0;
27 }
```

The terminal on the right shows the output of the program:

```
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code g++ proj.cpp &&
./a.out
Base Function
emon fedora ~/code
```

Img-1

Here(above), Base type pointer points to the Derived object “derived1”. When we call the `print()` function using `ptr`, it calls the member function from `Base`. Without virtual, C++ uses **static binding** (also called compile-time binding). The compiler looks at the **type of the pointer** (`Base*`) and decides at compile time: “this pointer is `Base*`, so call `Base::print()`.” It doesn't matter what object the pointer actually points to at runtime — the pointer's declared type wins.

In order to override the `Base` function instead of accessing it, we need to use **virtual functions** in the `Base` class(see below example).

(with virtual function)

**** N.B: This is the example of polymorphism(runtime)**

```
7 class Base {
8 public:
9     // Virtual function
10    virtual void display() {
11        cout << "Base function";
12    }
13 };
14
15 class Derived : public Base {
16 public:
17
18     // Overriding the base class function
19    void display() override {
20        cout << "Derived function";
21    }
22 };
23
24 int main() {
25     Base* Ptr;
26     Derived derived1;
27     Ptr = &derived1;
28     Ptr->display();
29     return 0;
30 }
```

emon fedora ~/code
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code g++ proj.cpp && ./a.out
Derived function
emon fedora ~/code

Img-2

With virtual, C++ uses **dynamic binding** (runtime binding). The compiler inserts a lookup through a **vtable** (virtual table) — a hidden table of function pointers attached to the object — so the decision of which function to call is deferred to runtime, based on the **actual type of the object**, not the pointer type.

`override` doesn't change runtime behaviour at all — it's a **compile-time safety check**. It tells the compiler "I intend for this function to override a virtual function from the base class." If you make a typo in the signature (wrong parameters, wrong const-ness, etc.), the compiler will throw an error instead of silently creating a new, unrelated function.

So what is virtual?

It tells C++:

"Don't decide which function to call until runtime."

Without virtual, C++ chooses the function based on the **type of the pointer/reference** (compile time).

With virtual, C++ chooses the function based on the **actual type of the object** (runtime).

So, **virtual enables runtime polymorphism**, while **override is a safety feature** that helps ensure your derived function really overrides the intended virtual function.

	No virtual	virtual
Binding time	Compile-time (static)	Runtime (dynamic)
Decided by	Pointer/reference type	Actual object type
Mechanism	Direct function call	Vtable lookup
Term	Function hiding	Function overriding

Virtual Functions and Function Overriding: A virtual function is a member function in the base class that we expect to redefine in derived classes. The `print()` method in the Derived class shadows the `print()` method in the Base class. However, if we create a pointer of Base type to point to an object of Derived class and call the `print()` function, it calls the `print()` function of the Base class. To avoid this, we declare the `print()` function of the Base class as virtual by using the `virtual` keyword.

If the virtual function is redefined in the derived class, the function in the derived class is executed even if it is called using a pointer of the base class object pointing to a derived class object. In such a case, the function is said to be overridden.

Size of Polymorphic object: Below image with polymorphism(dynamic binding)

```

10 class Base {
11     public:
12     int a;
13     virtual void print() {
14         cout << "Base Function" << endl;
15     }
16 };
17
18 class Derived : public Base {
19     public:
20     void print() {
21         cout << "Derived Function" << endl;
22     }
23 };
24
25 int main() {
26     Derived derived1;
27     Base* base1 = &derived1;
28     cout<<"Size of Obj: "<<sizeof(derived1);
29     return 0;
30 }

```

Terminal Output:

```

emon > fedora > ~/code
emon > fedora > ~/code
emon > fedora > ~/code g++ proj.cpp &
& ./a.out
Size of Obj: 16
emon > fedora > ~/code

```

Below image without polymorphism(only removed “virtual” keyword)

```

10 class Base {
11     public:
12     int a;
13     void print() {
14         cout << "Base Function" << endl;
15     }
16 };
17
18 class Derived : public Base {
19     public:
20     void print() {
21         cout << "Derived Function" << endl;
22     }
23 };
24
25 int main() {
26     Derived derived1;
27     Base* base1 = &derived1;
28     cout<<"Size of Obj: "<<sizeof(derived1);
29     return 0;
30 }

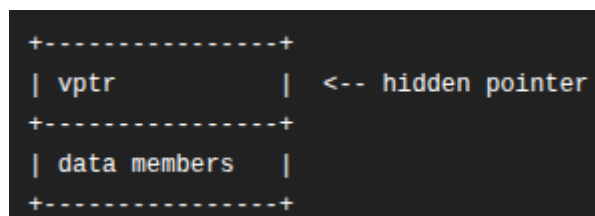
```

emon fedora ~/code
 ○ emon fedora ~/code
 ○ emon fedora ~/code
 ● emon fedora ~/code g++ proj.cpp &
 & ./a.out
 Size of Obj: 4
 ○ emon fedora ~/code

Above 2 image we can see, Size are different.

A polymorphic object is usually larger than a non-polymorphic object. The reason is that the compiler adds a hidden pointer called the **vp_{tr}** (virtual pointer). A class becomes polymorphic if it has at least one virtual function.

Each object of a polymorphic class contains a hidden pointer:



The vp_{tr} points to a **virtual table (vtable)**, which stores the addresses of the class's virtual functions. Without the vp_{tr}, the program would have no way to determine which virtual function should be called at runtime.

In short, compiler need to track the information that allows to dynamically resolve virtual function calls.

Why 16?

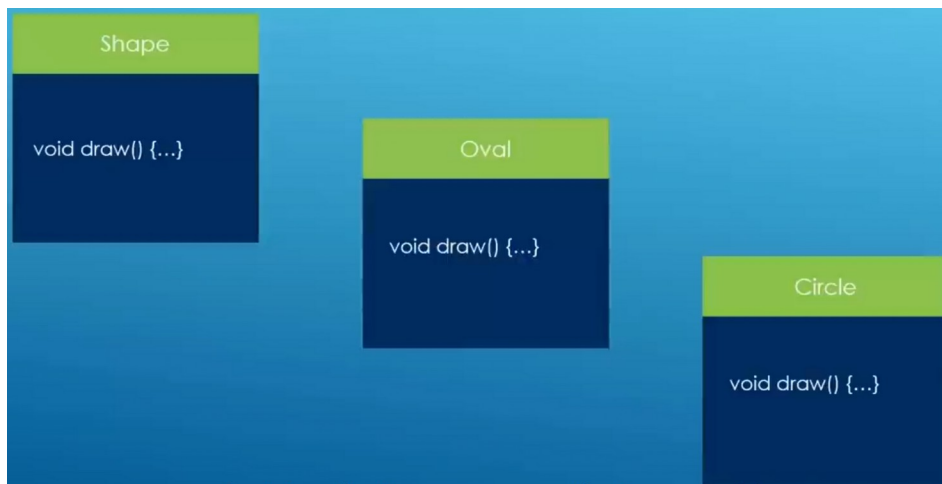
- `vptr` = 8 bytes
- `int` = 4 bytes
- Padding = 4 bytes (for 8-byte alignment)

Total = 16 bytes

Is there one vp_{tr} or one vtable?

- Each object has its own vp_{tr}.
- Each class has one vtable (in the common case).

Slicing:



In the above image, "Shape" is base class. Oval class is inheriting Shape class publicly, Circle class is inheriting Oval class publicly.

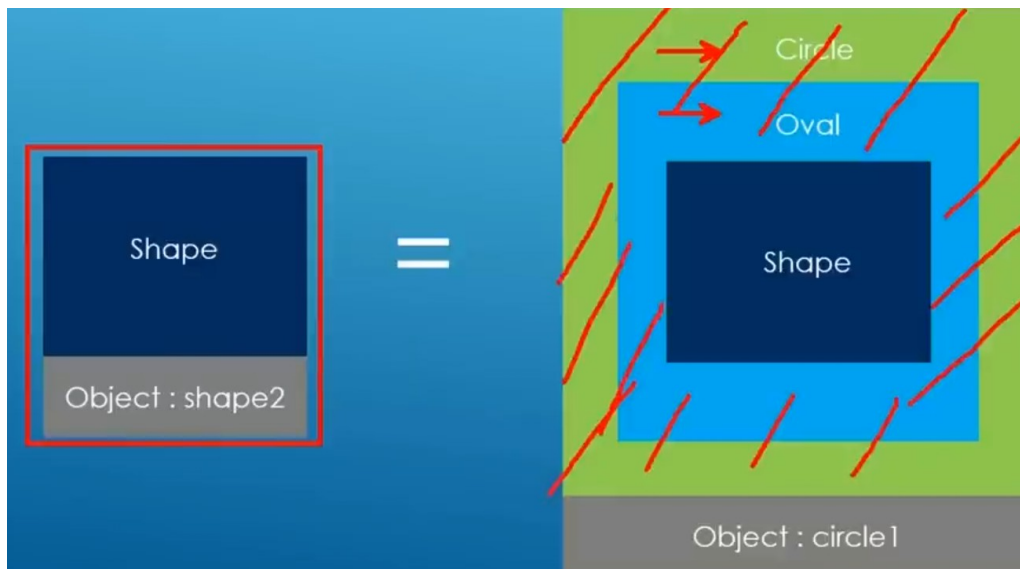
```
Slicing

Shape shape1("Shape1");
Oval oval1(2.0,3.5,"Oval1");
Circle circle1(3.3,"Circle1");

Shape shape2 = circle1;
```

In above image(line: 4), we are assigning a circle object to a shape object.(here not using pointer/reference)

Here compiler will notice that circle object has a part of shape class. So it will slicing off the extra part of Circle object except shape class part. And then assign shape object variable and that we are going to get in memory. This because, compiler will see this variable(shape2) has no enough memory to store all data of the circle object. Explained in the below image,



```

6  class Base {
7  protected:
8      int i;
9  public:
10     Base(int a) { i = a; }
11     virtual void display(){
12         cout << "I am Base class object, i = " << i << endl;
13     }
14 };
15
16 class Derived : public Base {
17     int j;
18 public:
19     Derived(int a, int b): Base(a){
20         j = b;
21     }
22     virtual void display()
23     {
24         cout << "I am Derived class object, i = " << i
25         << ", j = " << j << endl;
26     }
27 };
28 int main()
29 {
30     Base b(33);
31     Derived d(45, 54);
32     Base base1=d;
33     base1.display();
34     return 0;
35 }

```

emon fedora ~/code
emon fedora ~/code
emon fedora ~/code g++ proj.cpp & & ./a.out
I am Base class object, i = 45
emon fedora ~/code

In the above example derived object takes 2 parameters(a and b). But due to slicing, we only get Base class part, where i=45.

Polymorphic Object Stored in collections:

```

Objects sliced off!

Circle circle1(7.2,"circle1");
Oval oval1(13.3,1.2,"Oval1");
Circle circle2(11.2,"circle2");
Oval oval2(31.3,15.2,"Oval2");
Circle circle3(12.2,"circle3");
Oval oval3(53.3,9.2,"Oval3");

Shape shapes1[] {circle1,oval1,circle2,oval2,circle3,oval3};

```


Here(above) Storing the derived object in Shape class type array. So extra part of the objects will be slice off. In the array, only Shape part of the objects will be stored in the array. Also here copies of the object is stored, not reference.

Storing in references : Won't compile

```
Circle circle1(7.2,"circle1");
Oval oval1(13.3,1.2,"Oval1");
Circle circle2(11.2,"circle2");
Oval oval2(31.3,15.2,"Oval2");
Circle circle3(12.2,"circle3");
Oval oval3(53.3,9.2,"Oval3");

//This won't compile
const Shape& shapes2[] {circle1,oval1,circle2,oval2,circle3,oval3};
```

"shapes2[]" is shape type array of reference. C++ **does not allow arrays of references**. Because reference is just another name of a variable. And array is designed to modify data which is stored. But reference can't be modified or changes.

But Store in pointers works also no slicing off. Means all will be stored. (below)

Storing in pointers : Works

```
Circle circle1(7.2,"circle1");
Oval oval1(13.3,1.2,"Oval1");
Circle circle2(11.2,"circle2");
Oval oval2(31.3,15.2,"Oval2");
Circle circle3(12.2,"circle3");
Oval oval3(53.3,9.2,"Oval3");

Shape* shapes3[] { &circle1,&oval1,&circle2,&oval2,&circle3,&oval3};

for ( Shape* shape_ptr : shapes3){
    std::cout << "Inside array , sizeof(*shape_ptr) : " << sizeof(*shape_ptr) << std::endl;
    shape_ptr->draw();
    std::cout << std::endl;
}
```

Draw(); method will be called in polymorphic way.

Also works for Smart Pointer as below

Storing in smart pointers : Works

```
Circle circle1(7.2,"circle1");
Oval oval1(13.3,1.2,"Oval1");
Circle circle2(11.2,"circle2");
Oval oval2(31.3,15.2,"Oval2");
Circle circle3(12.2,"circle3");
Oval oval3(53.3,9.2,"Oval3");

→ std::shared_ptr<Shape> shapes4[] {std::make_shared<Circle>(12.2,"Circle4"),
                                     std::make_shared<Oval>(10.0,20.0,"Oval4")};

for(auto& s : shapes4){
    s->draw();
}
```

Inheritance & Polymorphism with Static Variable: See 1st [Static Variable](#)

1. Static class Member: A static data member is shared by **all objects** of the class — there's only one copy, not one per object.

```
#include <iostream>
using namespace std;

class Student {
public:
    static int totalStudents; // declaration
    Student() {
        totalStudents++;
    }
};

int Student::totalStudents = 0; // definition (required outside class, C++11 and ear

int main() {
    Student s1, s2, s3;
    cout << "Total students: " << Student::totalStudents << endl; // 3
    return 0;
}
```

Note: since C++17, you can use `inline static` to define and initialize it directly inside the class:

```
cpp

class Student {
public:
    inline static int totalStudents = 0; // no need for separate definition
};
```

Another Example:

```
7 class Shape{
8 public:
9     inline static int obj_count; // use "inline" if C++17 or later
10    Shape(string_view objDes): obj_description(objDes){
11        obj_count++;
12    }
13    virtual void draw(){
14        cout<<"Shape: Draw of "<<obj_description<<"obj count: "<<obj_count;NL;
15    }
16 protected:
17    string obj_description;
18 };
19 class Ellipse : public Shape{
20 public:
21     // inline static int obj_count=0;
22     Ellipse(string_view objDes):Shape(objDes){
23         // obj_count++;
24     }
25     virtual void draw(){
26         cout<<"Ellipse: Draw of "<<obj_description<<" obj count: "<<obj_count;NL;
27     }
28 };
29 int main(){
30     Shape shape1("Shape");
31     cout<<Shape::obj_count;NL;
32     Shape shape2("Shape");
33     cout<<Shape::obj_count;NL;
34     Shape shape3("Shape");
35     cout<<Shape::obj_count;NL;
36
37     Ellipse el1("Ellipse");
38     cout<<Shape::obj_count;NL;
39     Ellipse el2("Ellipse");
40     cout<<Shape::obj_count;NL;
41     Ellipse el3("Ellipse");
42     cout<<Shape::obj_count;NL;
43 }
```

emon fedora ~/code
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code g++ proj.cpp
\$./a.out
1
2
3
4
5
6
emon fedora ~/code

Here(above) Ellipse is a Shape. So total object is 6. But in below code... We can also track only ellipse object count by same variable(obj_count).

```

7  class Shape{
8      public:
9          inline static int obj_count; // use "inline" if C++17 or later
10         Shape(string_view objDes): obj_description(objDes){
11             obj_count++;
12         }
13         virtual void draw(){
14             cout<<"Shape: Draw of "<<obj_description<<"obj count: "<<obj_count<<NL;
15         }
16     protected:
17         string obj_description;
18 };
19 class Ellipse : public Shape{
20     public:
21         inline static int obj_count=0;
22         Ellipse(string_view objDes):Shape(objDes){
23             obj_count++;
24         }
25         virtual void draw(){
26             cout<<"Ellipse: Draw of "<<obj_description<<" obj count: "<<obj_count<<NL;
27         }
28 };
29 int main(){
30     Shape shape1("Shape");
31     cout<<"Shape: "<<Shape::obj_count<<NL;
32     Shape shape2("Shape");
33     cout<<"Shape: "<<Shape::obj_count<<NL;
34     Shape shape3("Shape");
35     cout<<"Shape: "<<Shape::obj_count<<NL<<NL;
36
37     Ellipse el1("Ellipse");
38     cout<<"Ellipse: "<<Ellipse::obj_count<<NL;
39     Ellipse el2("Ellipse");
40     cout<<"Ellipse: "<<Ellipse::obj_count<<NL;
41     Ellipse el3("Ellipse");
42     cout<<"Ellipse: "<<Ellipse::obj_count<<NL<<NL;
43
44     cout<<"Total Shape: "<<Shape::obj_count<<NL;
45     cout<<"Total Ellipse: "<<Ellipse::obj_count<<NL;
46 }

```

```

emon fedora ~/code
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code
emon fedora ~/code g++ proj.cpp
$&& ./a.out
Shape: 1
Shape: 2
Shape: 3

Ellipse: 1
Ellipse: 2
Ellipse: 3

Total Shape: 6
Total Ellipse: 3
emon fedora ~/code

```

We can see obj count(obj_count) variables are same in both class. Now Ellipse object count variable is going to maintain its own static variable and Shape also.

Shape::obj_count and Ellipse::obj_count are two completely different static variables. They just happen to have the same name. These are two different variables stored at different memory locations.

4. Static Member Function: Can be called without creating an object, and can only access static members (no access to this or non-static members).

```

class Student {
public:
    static int totalStudents;
    static void showCount() { // static member function
        cout << "Total: " << totalStudents << endl;
    }
};

int Student::totalStudents = 5;

int main() {
    Student::showCount(); // called directly via class name
    return 0;
}

```

“Final” in Inheritance: “Final” Key allow us to Restrict

1. Final virtual function/ function (override method in derived class)
2. Final Class(base class)

1. Final virtual function/ function:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() final {
        cout << "Shape::draw()\n";
    }
};

class Circle : public Shape {
public:
    void draw() override { // Error
        cout << "Circle::draw()\n";
    }
};
```

Shape::draw() method is final. Means no downstream class(derived class: Circle) can override this method. This is used to restrict, how we can override our own virtual methods in downstream classes(derived class).

2. Final Class: A class marked `final` **cannot be inherited**.

```
⌞ C++
class Director final : public Manager
{
};

Now nobody can create

⌞ C++
class CEO : public Director
{
};

because Director is final.
```

Above, Director is final. So Director class can't be inherited.

Virtual functions with Default Argument:

```
virtual double add(double a = 5 , double b = 5) const{
    std::cout << "Base::add() called" << std::endl;
    return (a + b + 1);
}
```

Here a=5, b=5 are default argument. Things to remember(below)

- Default arguments are handled at compile time
- Virtual functions are called at run time with polymorphism
- If you use default arguments with virtual functions, you might get weird(erroneous) results with polymorphism.

See Videos for details: 29:09:17 hour https://www.youtube.com/watch?v=8jLOx1hD3_o

It is recommended not to use default arguments.

Virtual Destructor: Remember previous example. Ellipse class inherit Shape class publicly. So if we create a Ellipse class object and when its times to destroy object, the order of the destructor call is: 1st Ellipse class destructor called then Shape class destructor called.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define NL printf("\n")
4  //g++ proj.cpp && ./a.out
5
6
7  class Shape{
8  public:
9      ~Shape(){
10         cout<<"Shape destructor Called";NL;
11     }
12     int a;
13     virtual void draw(){
14         cout<<"Shape: Draw of "<<obj_description;NL;
15     }
16 protected:
17     string obj_description;
18 };
19 class Ellipse : public Shape{
20 public:
21     ~Ellipse(){
22         cout<<"Ellipse destructor Called";NL;
23     }
24     Ellipse(string_view info){
25         this->obj_description=info;
26     }
27     virtual void draw(){
28         cout<<"Ellipse: Draw of "<<obj_description;NL;
29     }
30 };
31 int main(){
32     Ellipse ell("Ellipse");
33 }

```

emon fedora ~/code g++ proj.cpp && ./a.out
Ellipse destructor Called
Shape destructor Called

Problem is arise when we use base pointer to manage derived class objects. How?

```

31 int main(){
32     Shape* shapel=new Ellipse("Ellipse type");
33     delete shapel;
34 }

```

emon fedora ~/code g++ proj.cpp && ./a.out
Shape destructor Called

So derived class object destructor not called.!! Because our destructor is not virtual. The compiler is going to use static binding by seeing Shape type of the pointer, and decide at compile time which destructor to called.

So after using virtual destructor, this will show polymorphic behaviour, and call the actual type of the object destructor. And release the memory. As below


```

7 class Shape{
8 public:
9     virtual ~Shape(){
10         cout<<"Shape destructor Called";NL;
11     }
12     int a;
13     virtual void draw(){
14         cout<<"Shape: Draw of "<<obj_description;NL;
15     }
16 protected:
17     string obj_description;
18 };
19 class Ellipse : public Shape{
20 public:
21     ~Ellipse(){
22         cout<<"Ellipse destructor Called";NL;
23     }
24     Ellipse(string_view info){
25         this->obj_description=info;
26     }
27     virtual void draw(){
28         cout<<"Ellipse: Draw of "<<obj_description;NL;
29     }
30 };
31 int main(){
32     Shape* shape1=new Ellipse("Ellipse type");
33     delete shape1;
34 }

```

emon fedora ~/code
 g++ proj.cpp
 ./a.out
 Ellipse destructor Called
 Shape destructor Called

Remember, without virtual destructor, any kind of dynamic memory(not the shape part) we might have allocated in the constructor for Ellipse is going to be leaked out. Here, meaning Ellipse part which will use in dynamic memory allocation. Suppose a variable of Ellipse class part(not inherited part).

See Link(virtual destructor) for more... <https://www.programiz.com/cpp-programming/virtual-functions>

Dynamic Cast: dynamic_cast is a C++ casting operator used to **safely convert pointers or references within an inheritance hierarchy at runtime**.

In some cases we need to call/access non-polymorphic methods of the derived class and we have a pointer of base class pointing to the derived class objects. Then the pointer has no idea about non-polymorphic methods.

```

7 class base{
8 public:
9     base()=default;
10    virtual void speak(){
11        cout<<"This is base.";NL;
12    }
13 };
14 class derived:public base{
15 public:
16     derived()=default;
17     virtual void speak(){
18         cout<<"This is derived";NL;
19     }
20     void nonPoly(){
21         cout<<"This is non-polymorphic method";
22     }
23 };
24
25
26 int main() {
27     base* ptr=new derived();
28     ptr->speak();
29     ptr->nonPoly();
30 }

```

On the above example, see :line 29. Error.

Below we can access such methods by using `Dynamic_cast`

```
5  class base{
6  public:
7      base()=default;
8      virtual void speak(){
9          cout<<"This is base."<<NL;
10     }
11 };
12 class derived:public base{
13 public:
14     derived()=default;
15     virtual void speak(){
16         cout<<"This is derived"<<NL;
17     }
18     void nonPoly(){
19         cout<<"This is non-polymorphic method";
20     }
21 };
22
23 int main() {
24     base* ptr=new base();
25     ptr->speak();
26     // ptr->nonPoly();
27     derived* dptr= dynamic_cast<derived*>(ptr);
28     dptr->nonPoly();
29
30 }
```

`dynamic_cast` safely converts a base class pointer to a derived class pointer at runtime. If the conversion is valid, it returns the converted pointer; otherwise, it returns `nullptr`.

"ptr" is a base class pointer pointing to a `Derived1` object. `dynamic_cast` checks the actual object type at runtime. Since the object is of type `Derived1`, the cast succeeds. A valid `Derived*` pointer is returned.

For downcasting to work, the base class must be polymorphic. A class becomes polymorphic when it contains at least one virtual function.

If the base class doesn't contain any virtual function then, it is not polymorphic. As a result, `dynamic_cast` cannot perform runtime type checking and compilation fails.

*** Requires the base class to have at least one virtual function to apply `Dynamic_cast`

Syntax

```
dynamic_cast<new_type>(expression)
```

Parameters

- **new_type:** Target type for the conversion.
- **expression:** Pointer or reference being converted.

Return Value

- Returns a valid pointer if the cast succeeds.
- Returns nullptr if a pointer cast fails.
- Throws std::bad_cast if a reference cast fails.

It is required to check whether, `dynamic_cast` successful or not. Like below:

```
Derived1* dp2 = dynamic_cast<Derived1*>(bp);  
if (dp2 == nullptr)  
    cout << "Casting Failed" << endl;  
else  
    cout << "Casting Successful" << endl;
```

When Casting will failed:

```
5  class Base {  
6      virtual void print() {  
7          cout << "Base" << endl;  
8      }  
9  };  
10 class Derived1 : public Base {  
11     void print() {  
12         cout << "Derived1" << endl;  
13     }  
14 };  
15 class Derived2 : public Base {  
16     void print() {  
17         cout << "Derived2" << endl;  
18     }  
19 };  
20  
21 int main() {  
22     Derived1 d1;  
23     Base* bp = &d1;  
24     Derived2* dp2 = dynamic_cast<Derived2*>(bp);  
25     if (dp2 == nullptr)  
26         cout << "Casting Failed" << endl;  
27     else  
28         cout << "Casting Successful" << endl;  
29 }
```



The screenshot shows a terminal window with the following output:

```
g++ proj.cpp &&  
./a.out  
Casting Failed
```

`bp` points to a `Derived1` object. The code attempts to convert it to `Derived2*`. Since the object is not of type `Derived2`, the conversion is invalid. `dynamic_cast` returns `nullptr`.

Like pointer we can use references. As below:

```
7  class base{
8      public:
9          base()=default;
10         virtual void speak(){
11             cout<<"This is base."<<NL;
12         }
13     };
14     class derived:public base{
15     public:
16         derived()=default;
17         virtual void speak(){
18             cout<<"This is derived"<<NL;
19         }
20         void nonPoly(){
21             cout<<"This is non-polymorphic method"<<NL;
22         }
23     };
24
25
26     int main() {
27         derived d1;
28         base& ref=d1;
29         derived& dref=dynamic_cast<derived&>(ref);
30         dref.nonPoly();
31         dref.speak();
32     }
```

emon fedora ~/code

emon fedora ~/code

emon fedora ~/code

emon fedora ~/code

emon fedora ~/code g++ proj.cpp && ./a.out

This is non-polymorphic method

This is derived

emon fedora ~/code

When Casting fails?

`dynamic_cast` can fail when the **actual object at runtime is not compatible with the type you're trying to cast to**.

```
<> C++

class Animal {
public:
    virtual ~Animal() {}
};

class Dog : public Animal {};

class Cat : public Animal {};
```

Now:

```
<> C++

Animal* a = new Cat;

Dog* d = dynamic_cast<Dog*>(a);
```

Here, pointer type is animal, object type is cat. We want to cast to dog. But Dog is not a cat, means Dog is not derived class of cat. So casting fails.

In terms of reference, references cannot be nullptr. Instead, `dynamic_cast` throws: `"std::bad_cast"`

In case of checking casting fails or not we use if,else for pointer. But for reference we can use try catch since it throws an exception, like below:

```

</> C++

try {
    Dog& dog = dynamic_cast<Dog&>(animal);
}
catch (const bad_cast& e) {
    cout << "Cast failed";
}

```

While polymorphism works, see how constructor and destructor called. Below:

```
Base * b = new Derived;
```

Constructor and destructor call order :

- . Base Constructor
- . Derived Constructor
 - //Use constructed object
- . Derived Destructor
- . Base destructor

1. Base Constructor Called
2. Derived Constructor Called
3. Derived Destructor Called
4. Base Destructor Called

***Never call virtual(Polymorphic) function from constructor and destruction. To know details See: 30:08:23 Hour.(on the video)

- Calling a virtual function from a constructor or destructor won't give you polymorphic results
- The call will never go to a more derived class than the currently executing constructor or destructor
- In other words you will get static binding results

Pure Virtual function & Abstract Class:

Pure Virtual function: A **pure virtual function** is a virtual function that the base class declares but does not provide a required implementation. It forces any child class to write its own version of this function.

```
virtual void draw() = 0;
```

The " = 0 " is what makes it **pure virtual**.

It basically says, "Every derived class must provide its own draw()."

```
26 class Shape {
27 public:
28     virtual void draw() = 0;
29 };
30
31 class Circle : public Shape {
32 public:
33     void draw() override {
34         cout << "Drawing Circle";
35     }
36 };
```

Here, Circle provides an implementation for draw().

Abstract Class: A class containing **at least one pure virtual function** is called an **abstract class**.

cannot create an object of an abstract class. But we can create a pointer/reference:

***Derived class must provide the definition of pure virtual class. If a derived class doesn't provide it, then this derived class have to be Abstract class. Also can't create object of that derived class since this is also a abstract class.

We can provide a definition for a pure virtual function outside the class:

```
26 class Animal {
27 public:
28     virtual void sound() = 0;
29 };
30
31 void Animal::sound() {
32     cout << "Animal sound";
33 }
34
```

***Pure virtual means the function makes the class abstract; it does not necessarily mean the function can never have a definition.

Abstract class can the normal methods. And we call call them. Below: (Line 18)

```
7 class Animal {
8 public:
9     virtual void sound() = 0;
10 };
11
12 void Animal::sound() {
13     cout << "Animal sound";
14 }
15 class Dog : public Animal {
16 public:
17     void sound() override {
18         Animal::sound();
19         cout << " - Woof";
20     }
21 };
```

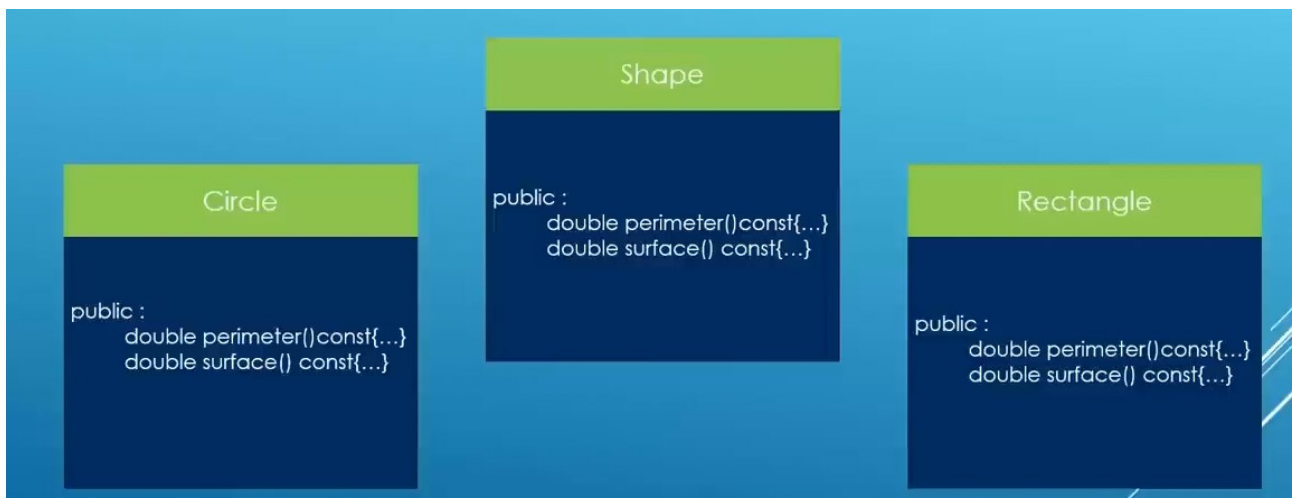
Also This way: below

```
7  class base{
8      public:
9          base()=default;
10         void speak(){
11             cout<<"This is base."<<NL;
12         }
13         virtual void draw() = 0;
14     };
15     class derived:public base{
16     public:
17         derived()=default;
18         virtual void draw(){
19             cout<<"This is derived"<<NL;
20         }
21     };
22
23     int main() {
24         derived d1;
25         base* ref=&d1;
26         ref->speak();
27     }
28 }
```

emon fedora ~/code
emon fedora ~/code g++ proj.cpp &&
./a.out
This is base.

* Abstract class can have both constructor and destructor.

Why need abstract class and pure virtual function?



Here(above), shape is a base class, and Circle, Rectangle is derived-class. Shape is abstract class by not defining the Perimeter and surface function. Derived must define the function.

Since shape class doesn't know which parameter need to define the perimeter and surface function. It let the derived to define cause the parameters needs to calculate the perimeter of Circle are diff then the perimeter of Rectangle. This is how abstract class and pure virtual function become useful.

Also a shape can be a Rectangle, Circle, Triangle etc. So let the derived-class to define what it needed.

Interface: An interface is a specification of something that will be fully implemented in a derived class but the specification itself resides in the abstract class.

An abstract class with only pure virtual functions and no member variable can be used as interfaces.

****Class can inherit many interfaces. _/_*